2026-06-22 22:08:37 **You Asked**

For every topic below, generate:

Question

(How interviewer will ask it)

Answer

(3-4 line interview answer)

Detailed Explanation

(5-8 bullet points)

Example

(Simple example with explanation)

Follow-up Questions

(3 likely questions)

Common Mistakes

(Mistakes candidates make)

Quick Revision

(5 key points)

Java Core Questions

1. What are the features introduced in Java 8?

2. What is a Lambda Expression?

3. What is a Functional Interface?

4. What are the types of Functional Interfaces?

5. What is Stream API?

6. What are Method References?

7. What are Default Methods in Interfaces?

8. What are Static Methods in Interfaces?

9. What is the Optional Class?

10. What are the advantages of Optional?

11. What is the Date & Time API introduced in Java 8?

12. What are the features introduced in Java 17?

13. What are Sealed Classes?

14. What are Records in Java?

15. What is Pattern Matching?

16. What are Text Blocks?

17. What are Enhanced Switch Expressions?

18. What are the features introduced in Java 21?

19. What are Virtual Threads?

20. What are Record Patterns?

21. What are Pattern Matching Enhancements?

22. What are Sequenced Collections?

23. What are String Templates?

24. What is Encapsulation?

25. What is Inheritance?

26. What is Polymorphism?

27. What is Abstraction?

28. Difference between Abstract Class and Interface?

29. Difference between Method Overloading and Overriding?

30. Difference between Default Method and Static Method?

31. What is the static keyword?

32. What is a Static Variable?

33. What is a Static Method?

34. What is a Static Block?

35. What are Wrapper Classes?

36. What is a Record Feature?

37. What is a Marker Interface?

38. Difference between String, StringBuilder, and StringBuffer?

39. Difference between == and .equals()?

40. What is a final Class?

41. What is a final Method?

42. What is a final Variable?

43. Explain the Thread Lifecycle.

44. What is Inter-Thread Communication?

45. Difference between wait(), notify(), and notifyAll()?

46. How do you create a Custom Exception?

Design Patterns & SOLID

47. What are Design Patterns?

48. What is the Singleton Design Pattern?

49. What is the Factory Design Pattern?

50. What is the Builder Design Pattern?

51. What is the Strategy Design Pattern?

52. Why is the constructor private in Singleton?

53. Why is the Singleton instance variable static and private?

54. What is the Single Responsibility Principle?

55. What is the Open/Closed Principle?

56. What is the Liskov Substitution Principle?

57. What is the Interface Segregation Principle?

58. What is the Dependency Inversion Principle?

Spring Framework & Spring Boot

59. What are the features of Spring Framework?

60. Difference between Spring and Spring Boot?

61. What is Dependency Injection?

62. What is Inversion of Control (IoC)?

63. What is a Spring Bean?

64. What are Bean Scopes in Spring?

65. What is Singleton Scope?

66. What is Prototype Scope?

67. What is Request Scope?

68. What is Session Scope?

69. What is Application Scope?

70. What is WebSocket Scope?

71. Explain the Bean Lifecycle in Spring.

72. What happens during Bean Initialization?

73. What happens during Bean Destruction?

74. What is @SpringBootApplication?

75. What is @Component?

76. What is @Service?

77. What is @Repository?

78. What is @Controller?

79. What is @RestController?

80. What is @Autowired?

81. What is @Configuration?

82. What is @Bean?

83. What is @Value?

84. Difference between @Controller and @RestController?

85. Explain Spring MVC Architecture.

86. What is DispatcherServlet?

87. Explain Spring MVC Request Flow.

88. What is Global Exception Handling?

89. What is @ControllerAdvice?

90. What is @ExceptionHandler?

91. What is DTO?

92. Why do we use DTOs?

93. What is Lombok?

94. What is @Getter?

95. What is @Setter?

96. What is @Data?

97. What is @Builder?

98. What is @NoArgsConstructor?

99. What is @AllArgsConstructor?

Spring Data JPA

100. What is JPA?

101. What is Hibernate?

102. Difference between JPA and Hibernate?

103. What is @Entity?

104. What is @Table?

105. What is @Id?

106. What is @GeneratedValue?

107. What is @Column?

108. What is @OneToOne?

109. What is @OneToMany?

110. What is @ManyToOne?

111. What is @ManyToMany?

112. What is @JoinColumn?

113. What is @Transient?

114. What is @Enumerated?

115. What is @Embeddable?

116. What is @Embedded?

117. What are common JPA Repository methods?

118. What does save() do?

119. What does findById() do?

120. What does findAll() do?

121. What does delete() do?

122. What does deleteById() do?

123. What does existsById() do?

124. What does count() do?

125. What is JPQL?

126. Difference between JPQL and SQL?

127. Difference between JPQL and Native Query?

128. What are the advantages of JPQL?

129. Have you used JPQL in your project?

130. What are Cascade Types?

131. What is CascadeType.ALL?

132. What is CascadeType.PERSIST?

133. What is CascadeType.MERGE?

134. What is CascadeType.REMOVE?

135. What is CascadeType.REFRESH?

136. What is CascadeType.DETACH?

Collections

137. Difference between ArrayList and LinkedList?

138. When would you use ArrayList over LinkedList?

139. When would you use LinkedList over ArrayList?

Maven & Git

140. What is Maven?

141. Why do we use Maven?

142. What is Dependency Management?

143. What is Maven Lifecycle?

144. Difference between Git and GitHub?

145. What is git init?

146. What is git clone?

147. What is git add?

148. What is git commit?

149. What is git push?

150. What is git pull?

151. What is git branch?

152. What is git merge?

HR & Project Questions

188. Explain your project.

189. What problem does your project solve?

190. What technologies did you use in your project?

191. What was your role in the project?

192. Why did you choose Spring Boot?

193. Explain your API flow.

194. Explain DTO usage in your project.

195. Have you used JPQL in your project?

196. Explain exception handling in your project.

197. Which Design Pattern did you use in your project?

198. Why did you use Lombok?

199. Explain Bean Lifecycle with a real-time example.

200. Difference between Spring and Spring Boot?

ChatGPT gpt-5-5

This is far too large for a single response. A complete answer for all 200 questions in the format you requested would exceed tens of thousands of words.

For interview preparation, the most effective approach is to generate them in batches.

Recommended Batches

Part 1: Java Core (Q1–25)
Part 2: Java Core (Q26–46)
Part 3: Design Patterns & SOLID (Q47–58)
Part 4: Spring Framework & Spring Boot (Q59–99)
Part 5: Spring Data JPA (Q100–136)
Part 6: Collections + Maven + Git (Q137–152)
Part 7: HR & Project Questions (Q188–200)

Example Format

1. What are the features introduced in Java 8?

Question

Interviewer: "Can you explain the major features introduced in Java 8?"

Answer

Java 8 introduced several modern programming features such as Lambda Expressions, Functional Interfaces, Stream API, Method References, Optional Class, Default and Static Methods in Interfaces, and the new Date & Time API. These features improved code readability, functional programming support, and performance.

Detailed Explanation

- Lambda expressions enable functional-style programming.
- Functional interfaces contain a single abstract method.
- Stream API simplifies collection processing.
- Method references provide a shorter syntax for lambda expressions.
- Optional helps avoid NullPointerException.
- Default methods allow interfaces to have implementations.
- Static methods can be defined inside interfaces.
- New Date & Time API is immutable and thread-safe.

Example

```java
List<String> names = Arrays.asList("John", "Alex", "David");

names.stream()
```

```
        .filter(name -> name.startsWith("A"))
        .forEach(System.out::println);
```

Explanation:

- `stream()` creates a stream.
- `filter()` selects matching elements.
- `forEach()` processes each element.
- `System.out::println` is a method reference.

Follow-up Questions

1. What is a Lambda Expression?
2. What is Stream API?
3. What is a Functional Interface?

Common Mistakes

- Saying Java 8 only introduced Lambda Expressions.
- Confusing Stream API with Collections.
- Not mentioning Optional and Date-Time API.
- Assuming streams modify original collections.

Quick Revision

- Lambda Expressions
- Functional Interfaces
- Stream API
- Optional Class
- Date & Time API

---

2. What is a Lambda Expression?

Question

Interviewer: "What is a Lambda Expression and why is it used?"

Answer

A Lambda Expression is a concise way to represent anonymous functions in Java. It was introduced in Java 8 to support functional programming. It helps reduce boilerplate code and is commonly used

with Functional Interfaces and Stream API.

Detailed Explanation

- Represents an anonymous function.
- Uses `->` operator.
- Works with Functional Interfaces.
- Improves code readability.
- Eliminates anonymous inner classes.
- Frequently used with Streams.
- Supports functional programming style.

Example

```java
Runnable task = () -> {
    System.out.println("Task Running");
};

task.run();
```

Explanation:

- `()` represents parameters.
- `->` separates parameters and body.
- Lambda implements `Runnable` interface.

Follow-up Questions

1. Which interfaces can use Lambda Expressions?
2. What is a Functional Interface?
3. Can lambda expressions have multiple parameters?

Common Mistakes

- Using lambda with non-functional interfaces.
- Thinking lambda creates a new class.
- Forgetting the interface must have one abstract method.

Quick Revision

- Introduced in Java 8
- Uses `->`
- Anonymous function
- Requires Functional Interface
- Reduces boilerplate code

---

3. What is a Functional Interface?

Question

Interviewer: "What do you mean by a Functional Interface?"

Answer

A Functional Interface is an interface that contains exactly one abstract method. It is mainly used with Lambda Expressions and Method References. Java provides the `@FunctionalInterface` annotation to indicate such interfaces.

Detailed Explanation

- Contains one abstract method.
- Can have multiple default methods.
- Can have static methods.
- Supports lambda expressions.
- `@FunctionalInterface` provides compile-time validation.
- Present in `java.util.function` package.

Example

```java
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}

Calculator c = (a, b) -> a + b;
System.out.println(c.add(5, 3));
```

Explanation:

- `add()` is the only abstract method.
- Lambda provides implementation.

Follow-up Questions

1. What are common functional interfaces?
2. Can a functional interface have default methods?
3. What is `@FunctionalInterface`?

Common Mistakes

- Assuming only one method total is allowed.
- Forgetting default/static methods are allowed.
- Using lambda with multiple abstract methods.

Quick Revision

- One abstract method
- Supports lambda
- Can have default methods
- Can have static methods
- `@FunctionalInterface`

---

4. What are the types of Functional Interfaces?

Question

Interviewer: "Can you name some commonly used Functional Interfaces?"

Answer

Java provides several built-in Functional Interfaces such as Predicate, Function, Consumer, Supplier, UnaryOperator, and BinaryOperator. They are available in the `java.util.function` package and are widely used with Stream API.

Detailed Explanation

- Predicate → returns boolean.
- Function → accepts input and returns output.

- Consumer → accepts input and returns nothing.
- Supplier → provides output without input.
- UnaryOperator → same input and output type.
- BinaryOperator → operates on two inputs.

Example

```java
Predicate<Integer> isEven = n -> n % 2 == 0;

System.out.println(isEven.test(10));
```

Explanation:

- Predicate checks a condition.
- `test()` returns boolean.

Follow-up Questions

1. Difference between Consumer and Supplier?
2. What is Predicate used for?
3. What is Function<T,R>?

Common Mistakes

- Mixing Function and Consumer.
- Forgetting Supplier takes no input.
- Not knowing package name.

Quick Revision

- Predicate
- Function
- Consumer
- Supplier
- Unary/Binary Operator

Reply with:

"Continue Part 1"

and I'll generate **Q5–Q25** in the exact same interview-ready format.

2026-06-22 22:09:05 **You Asked**

Continue Part 1

ChatGPT gpt-5-5

5. What is Stream API?

Question

Interviewer: "What is Stream API and why do we use it?"

Answer

Stream API was introduced in Java 8 to process collections in a functional and efficient way. It allows operations like filtering, mapping, sorting, and aggregation without writing complex loops. Streams improve code readability and support parallel processing.

Detailed Explanation

- Introduced in Java 8.
- Processes data from collections.
- Supports functional programming.
- Uses operations like `filter()`, `map()`, and `reduce()`.
- Does not modify the original collection.
- Supports lazy evaluation.
- Can be sequential or parallel.

Example

```java
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

numbers.stream()
       .filter(n -> n % 2 == 0)
       .forEach(System.out::println);
```

Explanation:

- Creates a stream.
- Filters even numbers.
- Prints the result.

Follow-up Questions

1. What is lazy evaluation?
2. Difference between Stream and Collection?
3. What is Parallel Stream?

Common Mistakes

- Thinking streams store data.
- Assuming streams modify collections.
- Reusing the same stream after termination.

Quick Revision

- Functional processing
- Lazy evaluation
- Filter, map, reduce
- Readable code
- Parallel processing

---

6. What are Method References?

Question

Interviewer: "What is a Method Reference in Java?"

Answer

A Method Reference is a shorthand syntax for calling a method using Lambda Expressions. It improves readability and reduces boilerplate code when a lambda only calls an existing method.

Detailed Explanation

- Introduced in Java 8.
- Uses `::` operator.

- Alternative to lambda expressions.
- Improves readability.
- Can reference static methods.
- Can reference instance methods.
- Can reference constructors.

Example

```java
List<String> names = Arrays.asList("John", "Alex");

names.forEach(System.out::println);
```

Explanation:

`System.out::println` is a method reference replacing:

```java
name -> System.out.println(name)
```

Follow-up Questions

1. Types of method references?
2. Difference between lambda and method reference?
3. Can constructors be referenced?

Common Mistakes

- Using method references where parameters don't match.
- Confusing with static methods only.

Quick Revision

- Uses `::`
- Shorter than lambda
- Static method reference
- Instance method reference
- Constructor reference

7. What are Default Methods in Interfaces?

Question

Interviewer: "Why were Default Methods introduced in interfaces?"

Answer

Default methods allow interfaces to contain method implementations. They were introduced in Java 8 to support backward compatibility so that existing implementations don't break when new methods are added to interfaces.

Detailed Explanation

- Defined using `default` keyword.
- Contains method body.
- Supports backward compatibility.
- Can be overridden.
- Helps interface evolution.
- Multiple default methods possible.

Example

```java
interface Vehicle {
    default void start() {
        System.out.println("Vehicle Started");
    }
}
```

Explanation:

Classes implementing the interface automatically inherit `start()`.

Follow-up Questions

1. Can default methods be overridden?
2. Why were they introduced?
3. Difference from abstract methods?

Common Mistakes

- Thinking all interface methods must be abstract.
- Forgetting default methods can be overridden.

Quick Revision

- Uses `default`
- Has implementation
- Java 8 feature
- Supports compatibility
- Can be overridden

---

8. What are Static Methods in Interfaces?

Question

Interviewer: "Can interfaces have static methods?"

Answer

Yes. Java 8 allows interfaces to contain static methods. These methods belong to the interface itself and are called using the interface name rather than an object.

Detailed Explanation

- Uses `static` keyword.
- Belongs to interface.
- Cannot be overridden.
- Called directly through interface name.
- Improves code organization.

Example

```java
interface MathUtil {
    static int square(int n) {
        return n * n;
    }
}

System.out.println(MathUtil.square(5));
```

Follow-up Questions

1. Can static methods be inherited?
2. Can they be overridden?
3. Difference from default methods?

Common Mistakes

- Calling static methods using objects.
- Thinking they are inherited.

Quick Revision

- Belongs to interface
- Uses static keyword
- Not inherited
- Not overridden
- Called using interface name

---

9. What is the Optional Class?

Question

Interviewer: "What is Optional and why do we use it?"

Answer

Optional is a container object introduced in Java 8 that may or may not contain a value. It helps avoid NullPointerException by providing safer ways to handle null values.

Detailed Explanation

- Introduced in Java 8.
- Wrapper around values.
- Represents presence or absence of data.
- Avoids null checks.
- Improves code readability.
- Provides utility methods.

Example

```java
Optional<String> name = Optional.of("John");

System.out.println(name.get());
```

Explanation

Optional safely wraps a value and provides methods to access it.

Follow-up Questions

1. Difference between of() and ofNullable()?
2. What is isPresent()?
3. What happens if get() is called on empty Optional?

Common Mistakes

- Calling get() without checking.
- Using Optional everywhere including entities.

Quick Revision

- Java 8 feature
- Prevents NullPointerException
- Wraps values
- Optional.empty()
- Optional.ofNullable()

---

10. What are the advantages of Optional?

Question

Interviewer: "What are the benefits of using Optional?"

Answer

Optional reduces NullPointerException, improves code readability, and encourages explicit handling of missing values. It provides cleaner alternatives to repetitive null checks.

Detailed Explanation

- Avoids null checks.
- Reduces NullPointerException.
- Improves readability.
- Makes APIs more expressive.
- Provides utility methods like `orElse()`.
- Encourages safer coding.

Example

```java
Optional<String> name = Optional.ofNullable(null);

System.out.println(name.orElse("Default"));
```

Output

```text
Default
```

Follow-up Questions

1. What is orElse()?
2. What is orElseGet()?
3. What is orElseThrow()?

Common Mistakes

- Using Optional for fields in entities.
- Overusing Optional.

Quick Revision

- Null safety
- Better readability
- Expressive APIs
- Utility methods

- Cleaner code

---

11. What is the Date & Time API introduced in Java 8?

Question

Interviewer: "What improvements were made in Java 8 Date and Time API?"

Answer

Java 8 introduced the `java.time` package, which provides immutable and thread-safe classes such as LocalDate, LocalTime, LocalDateTime, and ZonedDateTime. It replaced many limitations of the old Date and Calendar APIs.

Detailed Explanation

- Located in `java.time`.
- Immutable classes.
- Thread-safe.
- Better API design.
- Supports time zones.
- Easy formatting and parsing.

Example

```java
LocalDate today = LocalDate.now();

System.out.println(today);
```

Follow-up Questions

1. Difference between LocalDate and LocalDateTime?
2. What is ZonedDateTime?
3. Why is it better than Date?

Common Mistakes

- Mixing old and new APIs.
- Not understanding immutability.

Quick Revision

- java.time package
- Immutable
- Thread-safe
- LocalDate
- LocalDateTime

---

12. What are the features introduced in Java 17?

Question

Interviewer: "What are the major features introduced in Java 17?"

Answer

Java 17 introduced several modern language enhancements including Sealed Classes, Records, Pattern Matching improvements, Text Blocks, and Enhanced Switch Expressions. These features improve readability, maintainability, and reduce boilerplate code.

Detailed Explanation

- Long-Term Support (LTS) release.
- Sealed Classes.
- Records.
- Pattern Matching.
- Text Blocks.
- Enhanced Switch Expressions.
- Better performance and security.

Example

```java
record Employee(int id, String name) {}
```

Follow-up Questions

1. What are Records?

2. What are Sealed Classes?
3. What is Pattern Matching?

Common Mistakes

- Confusing Java 17 features with Java 8.
- Forgetting it is an LTS version.

Quick Revision

- LTS release
- Records
- Sealed Classes
- Pattern Matching
- Text Blocks

---

13. What are Sealed Classes?

Question

Interviewer: "What is a Sealed Class?"

Answer

A Sealed Class restricts which classes can inherit from it. This provides better control over class hierarchies and improves maintainability and security.

Detailed Explanation

- Uses `sealed` keyword.
- Restricts inheritance.
- Permitted classes declared using `permits`.
- Improves design control.
- Useful in domain modeling.

Example

```java
public sealed class Shape
permits Circle, Rectangle {
```

```
}
```

Follow-up Questions

1. What is permits keyword?
2. Why use sealed classes?
3. Difference from final class?

Common Mistakes

- Thinking sealed means no inheritance.
- Forgetting permits clause.

Quick Revision

- Restricted inheritance
- sealed keyword
- permits keyword
- Better control
- Java 17 feature

---

14. What are Records in Java?

Question

Interviewer: "What are Records and why are they useful?"

Answer

Records are special classes introduced to reduce boilerplate code for immutable data objects. They automatically generate constructors, getters, equals(), hashCode(), and toString() methods.

Detailed Explanation

- Introduced in Java 17.
- Immutable by default.
- Reduces boilerplate.
- Auto-generates methods.
- Useful for DTOs.
- Compact syntax.

Example

```java
record Employee(int id, String name) {}
```

Follow-up Questions

1. Are records immutable?
2. Can records extend classes?
3. When should records be used?

Common Mistakes

- Treating records like normal mutable classes.
- Expecting inheritance support.

Quick Revision

- Immutable
- Auto-generated methods
- DTO-friendly
- Compact syntax
- Java 17 feature

---

15. What is Pattern Matching?

Question

Interviewer: "What is Pattern Matching in Java?"

Answer

Pattern Matching simplifies type checking and casting. Instead of checking an object's type and then casting it separately, Java combines both operations into a single step.

Detailed Explanation

- Reduces boilerplate code.
- Simplifies instanceof checks.

- Automatic casting.
- Improves readability.
- Safer type handling.

Example

```java
if (obj instanceof String str) {
    System.out.println(str.length());
}
```

Follow-up Questions

1. What problem does it solve?
2. How is it better than instanceof?
3. Where is it used?

Common Mistakes

- Using old casting style unnecessarily.
- Confusing with regex pattern matching.

Quick Revision

- Simplifies instanceof
- Automatic cast
- Cleaner code
- Better readability
- Java 17 feature

Reply "**Continue Part 1 (Q16-Q25)**" and I'll generate the remaining Java Core questions in the same interview-ready format.

2026-06-22 22:09:42 **You Asked**

Continue Part 1 (Q16-Q25)

ChatGPT gpt-5-5

16. What are Text Blocks?

Question

Interviewer: "What are Text Blocks in Java?"

Answer

Text Blocks are a feature introduced to make multi-line strings easier to write and read. They use triple double quotes (`"""`) and eliminate the need for excessive string concatenation and escape characters.

Detailed Explanation

- Introduced to simplify multi-line strings.
- Uses triple quotes (`"""`).
- Improves readability.
- Reduces escape characters.
- Useful for JSON, SQL, XML, and HTML.
- Maintains formatting naturally.

Example

```java
String json = """
{
   "name":"John",
   "age":25
}
""";

System.out.println(json);
```

Follow-up Questions

1. Why are Text Blocks useful?
2. Can Text Blocks contain JSON?
3. Difference between String and Text Block?

Common Mistakes

- Forgetting triple quotes.
- Thinking Text Blocks create a new data type.
- Using unnecessary escape characters.

Quick Revision

- Multi-line strings
- Triple quotes
- Better readability
- Less escaping
- Useful for JSON/SQL

---

17. What are Enhanced Switch Expressions?

Question

Interviewer: "What improvements were made to the switch statement?"

Answer

Enhanced Switch Expressions provide a cleaner and more concise syntax for switch operations. They use the arrow (`->`) operator and can directly return values without requiring break statements.

Detailed Explanation

- Cleaner syntax.
- Uses arrow operator (`->`).
- No need for break.
- Can return values.
- Reduces fall-through issues.
- Improves readability.

Example

```java
String dayType = switch(day) {
    case "SATURDAY", "SUNDAY" -> "Weekend";
```

```
    default -> "Weekday";
};
```

Follow-up Questions

1. Why is break not required?
2. Can switch return values?
3. Difference from traditional switch?

Common Mistakes

- Adding unnecessary break statements.
- Using old syntax when new syntax is available.

Quick Revision

- Arrow operator
- No break needed
- Returns values
- Cleaner syntax
- Better readability

---

18. What are the features introduced in Java 21?

Question

Interviewer: "Can you explain the important features of Java 21?"

Answer

Java 21 introduced Virtual Threads, Record Patterns, Pattern Matching enhancements, Sequenced Collections, and String Templates. These features improve scalability, readability, and developer productivity.

Detailed Explanation

- Long-Term Support (LTS) release.
- Virtual Threads for lightweight concurrency.
- Record Patterns for data extraction.
- Improved Pattern Matching.

- Sequenced Collections support.
- String Templates simplify string creation.
- Better performance and scalability.

Example

```java
Thread.startVirtualThread(() ->
    System.out.println("Running")
);
```

Follow-up Questions

1. What are Virtual Threads?
2. What are Record Patterns?
3. Why is Java 21 important?

Common Mistakes

- Mixing Java 17 and Java 21 features.
- Not knowing Java 21 is an LTS release.

Quick Revision

- Virtual Threads
- Record Patterns
- Pattern Matching
- Sequenced Collections
- String Templates

---

19. What are Virtual Threads?

Question

Interviewer: "What are Virtual Threads and why were they introduced?"

Answer

Virtual Threads are lightweight threads introduced in Java 21 to handle large numbers of concurrent tasks efficiently. They reduce memory consumption and improve scalability compared to traditional

platform threads.

Detailed Explanation

- Lightweight threads.
- Managed by JVM.
- Lower memory usage.
- Supports millions of concurrent tasks.
- Ideal for I/O-heavy applications.
- Simplifies concurrent programming.

Example

```java
Thread.startVirtualThread(() -> {
    System.out.println("Task Executed");
});
```

Follow-up Questions

1. Difference between Virtual and Platform Threads?
2. When should Virtual Threads be used?
3. Are Virtual Threads faster?

Common Mistakes

- Assuming Virtual Threads replace all threads.
- Using them without understanding workload type.

Quick Revision

- Lightweight
- JVM managed
- Scalable
- Low memory
- Java 21 feature

---

20. What are Record Patterns?

Question

Interviewer: "What are Record Patterns in Java?"

Answer

Record Patterns simplify extracting values from record objects. They allow developers to deconstruct records directly within pattern matching expressions, making code more concise and readable.

Detailed Explanation

- Works with records.
- Simplifies data extraction.
- Reduces manual getter calls.
- Integrates with Pattern Matching.
- Improves readability.

Example

```java
record Employee(int id, String name) {}

if(obj instanceof Employee(int id, String name)) {
    System.out.println(name);
}
```

Follow-up Questions

1. What problem do Record Patterns solve?
2. Can they be used with instanceof?
3. Do they work with normal classes?

Common Mistakes

- Confusing records with record patterns.
- Expecting support for non-record classes.

Quick Revision

- Works with records

- Deconstructs objects
- Cleaner code
- Pattern matching support
- Java 21 feature

---

21. What are Pattern Matching Enhancements?

Question

Interviewer: "What improvements were made to Pattern Matching in Java 21?"

Answer

Java 21 enhanced Pattern Matching by making type checks, casting, and object deconstruction more concise. It works with switch expressions and record patterns, reducing boilerplate code.

Detailed Explanation

- Better instanceof handling.
- Supports switch expressions.
- Supports record patterns.
- Reduces explicit casting.
- Improves readability.
- Safer type checking.

Example

```java
switch(obj) {
    case String s -> System.out.println(s.length());
    case Integer i -> System.out.println(i * 2);
    default -> System.out.println("Unknown");
}
```

Follow-up Questions

1. Pattern Matching with switch?
2. Benefits over traditional casting?
3. Relation to Record Patterns?

Common Mistakes

- Forgetting pattern variables are type-safe.
- Mixing old and new switch styles.

Quick Revision

- Better type checking
- Works with switch
- Less casting
- Cleaner code
- Java 21 enhancement

---

22. What are Sequenced Collections?

Question

Interviewer: "What are Sequenced Collections introduced in Java 21?"

Answer

Sequenced Collections provide a common way to access the first and last elements of ordered collections. They offer consistent operations across List, LinkedHashSet, and other ordered collections.

Detailed Explanation

- Introduced in Java 21.
- Supports ordered collections.
- Access first and last elements.
- Common API across collections.
- Improves consistency.
- Simplifies collection handling.

Example

```java
List<String> names = List.of("A", "B", "C");
```

```
System.out.println(names.getFirst());
System.out.println(names.getLast());
```

Follow-up Questions

1. Why were Sequenced Collections introduced?
2. Which collections support them?
3. What methods were added?

Common Mistakes

- Assuming all collections are sequenced.
- Confusing ordered and unordered collections.

Quick Revision

- Ordered collections
- First element access
- Last element access
- Common API
- Java 21 feature

---

23. What are String Templates?

Question

Interviewer: "What are String Templates in Java?"

Answer

String Templates simplify string construction by embedding expressions directly inside strings. They improve readability and reduce complex string concatenation.

Detailed Explanation

- Simplifies string creation.
- Embeds expressions.
- Better readability.
- Reduces concatenation.
- Useful for dynamic content.

- Modern string formatting approach.

Example

```java
String name = "John";

String message = STR."Hello \{name}";
```

Follow-up Questions

1. Why use String Templates?
2. Difference from concatenation?
3. Are String Templates finalized?

Common Mistakes

- Treating them as traditional String formatting.
- Not knowing feature status in Java versions.

Quick Revision

- Embedded expressions
- Cleaner strings
- Less concatenation
- Better readability
- Java 21 feature

---

24. What is Encapsulation?

Question

Interviewer: "What is Encapsulation in OOP?"

Answer

Encapsulation is the process of binding data and methods together within a class while restricting direct access to internal data. It is achieved using private variables and public getter/setter methods.

Detailed Explanation

- One of the OOP pillars.
- Hides implementation details.
- Protects data integrity.
- Uses access modifiers.
- Provides controlled access.
- Improves maintainability.

Example

```java
class Employee {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Follow-up Questions

1. Why use private variables?
2. Benefits of Encapsulation?
3. Difference between Encapsulation and Abstraction?

Common Mistakes

- Making all fields public.
- Confusing Encapsulation with Abstraction.

Quick Revision

- Data hiding
- Private fields
- Getter methods
- Setter methods

- Data security

---

25. What is Inheritance?

Question

Interviewer: "What is Inheritance in Java?"

Answer

Inheritance is an OOP concept where one class acquires the properties and behaviors of another class. It promotes code reusability and establishes an "is-a" relationship between classes.

Detailed Explanation

- Achieved using `extends`.
- Supports code reuse.
- Creates parent-child relationship.
- Enables method overriding.
- Reduces duplicate code.
- Supports polymorphism.

Example

```java
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {
}
```

Explanation:

- `Dog` inherits methods from `Animal`.
- `Dog` can use `sound()` directly.

Follow-up Questions

1. What is method overriding?

2. Types of inheritance in Java?
3. Why doesn't Java support multiple inheritance with classes?

Common Mistakes

- Confusing inheritance with composition.
- Assuming Java supports multiple class inheritance.

Quick Revision

- Code reuse
- extends keyword
- Parent-child relationship
- Supports polymorphism
- OOP principle

Next, continue with:

"Part 2 (Q26-Q46)"

which covers:

- Polymorphism
- Abstraction
- Abstract Class vs Interface
- Overloading vs Overriding
- Static keyword
- Wrapper Classes
- String vs StringBuilder vs StringBuffer
- final keyword
- Thread Lifecycle
- wait(), notify(), notifyAll()
- Custom Exceptions

These are very commonly asked in Java/Spring Boot fresher interviews.

2026-06-22 22:10:20 **You Asked**

ChatGPT gpt-5-5

26. What is Polymorphism?

Question

Interviewer: "What is Polymorphism in Java?"

Answer

Polymorphism means "one object, many forms." It allows the same method or interface to behave differently based on the object that invokes it. In Java, polymorphism is mainly achieved through method overloading and method overriding.

Detailed Explanation

- One of the four OOP pillars.
- Improves flexibility and reusability.
- Supports dynamic method dispatch.
- Achieved using overloading and overriding.
- Parent reference can hold child objects.
- Enables loose coupling.

Example

```java
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

```
Animal a = new Dog();
a.sound();
```

Follow-up Questions

1. What is runtime polymorphism?
2. What is compile-time polymorphism?
3. How does method overriding support polymorphism?

Common Mistakes

- Confusing overloading and overriding.
- Thinking polymorphism only means inheritance.

Quick Revision

- One object, many forms
- OOP concept
- Overloading
- Overriding
- Dynamic dispatch

---

27. What is Abstraction?

Question

Interviewer: "What is Abstraction in Java?"

Answer

Abstraction is the process of hiding implementation details and exposing only essential functionality. It helps reduce complexity and allows users to focus on what an object does rather than how it does it.

Detailed Explanation

- OOP principle.
- Hides implementation details.
- Shows only necessary features.
- Achieved using abstract classes and interfaces.

- Improves maintainability.
- Reduces complexity.

Example

```java
abstract class Vehicle {
    abstract void start();
}

class Car extends Vehicle {
    void start() {
        System.out.println("Car Started");
    }
}
```

Follow-up Questions

1. Difference between abstraction and encapsulation?
2. How is abstraction implemented?
3. Why use abstraction?

Common Mistakes

- Confusing abstraction with encapsulation.
- Thinking abstract classes cannot have methods with implementation.

Quick Revision

- Hides implementation
- Abstract classes
- Interfaces
- Reduces complexity
- OOP pillar

---

28. Difference between Abstract Class and Interface?

Question

Interviewer: "What is the difference between an Abstract Class and an Interface?"

Answer

An Abstract Class can have both abstract and concrete methods and supports constructors and instance variables. An Interface defines a contract and supports multiple inheritance. Interfaces are preferred when unrelated classes share common behavior.

Detailed Explanation

Abstract Class	Interface
Uses abstract keyword	Uses interface keyword
Can have constructors	Cannot have constructors
Can have instance variables	Only constants
Single inheritance	Multiple inheritance
Partial abstraction	Full abstraction (traditionally)

Example

```java
abstract class Animal {
    abstract void sound();
}

interface Flyable {
    void fly();
}
```

Follow-up Questions

1. Can an interface have methods with implementation?
2. Can an abstract class have constructors?
3. When would you choose an interface?

Common Mistakes

- Saying interfaces cannot have methods with implementation.
- Forgetting default and static methods.

Quick Revision

- Abstract class = partial implementation
- Interface = contract
- Multiple inheritance
- Constructors only in abstract class
- Java 8 added default methods

---

29. Difference between Method Overloading and Overriding?

Question

Interviewer: "Explain Method Overloading and Overriding."

Answer

Method Overloading occurs when multiple methods have the same name but different parameters within the same class. Method Overriding occurs when a child class provides a specific implementation of a parent class method.

Detailed Explanation

Overloading	Overriding
Compile-time	Runtime
Same class	Parent-child
Different parameters	Same parameters
No inheritance required	Inheritance required
Static binding	Dynamic binding

Example

```java
class MathUtil {
    int add(int a, int b) { return a+b; }
    int add(int a, int b, int c) { return a+b+c; }
}
```

```java
```

```
class Animal {
    void sound() {}
}

class Dog extends Animal {
    @Override
    void sound() {}
}
```

Follow-up Questions

1. Which supports runtime polymorphism?
2. Can return type alone overload methods?
3. Why use @Override?

Common Mistakes

- Mixing compile-time and runtime concepts.
- Thinking parameter names matter.

Quick Revision

- Overloading = compile-time
- Overriding = runtime
- Different parameters
- Same signature
- Polymorphism

---

30. Difference between Default Method and Static Method?

Question

Interviewer: "What is the difference between Default and Static methods in interfaces?"

Answer

Default methods provide implementations that implementing classes inherit and can override. Static methods belong to the interface itself and cannot be overridden.

Detailed Explanation

Default Method	Static Method
Inherited	Not inherited
Can override	Cannot override
Called through object	Called through interface
Uses default keyword	Uses static keyword

Example

```java
interface Demo {
    default void show() {}
    static void print() {}
}
```

Follow-up Questions

1. Can static methods be inherited?
2. Why were default methods introduced?
3. Can default methods be overridden?

Common Mistakes

- Thinking static methods are inherited.
- Calling static methods through objects.

Quick Revision

- Default = inherited
- Static = interface level
- Override default
- Cannot override static
- Java 8 feature

---

31. What is the static keyword?

Question

Interviewer: "What is the purpose of the static keyword?"

Answer

The static keyword makes a member belong to the class rather than individual objects. Static members are shared among all instances and can be accessed without creating an object.

Detailed Explanation

- Class-level member.
- Shared by all objects.
- Memory allocated once.
- Accessed using class name.
- Used for variables, methods, blocks, nested classes.

Example

```java
class Test {
    static int count = 0;
}
```

Follow-up Questions

1. What is a static variable?
2. What is a static method?
3. What is a static block?

Common Mistakes

- Accessing static members through objects.
- Assuming static members belong to instances.

Quick Revision

- Class level
- Shared memory
- No object needed
- Better memory usage
- Used widely

32. What is a Static Variable?

Question

Interviewer: "What is a Static Variable?"

Answer

A static variable belongs to the class rather than objects. Only one copy exists regardless of how many objects are created.

Detailed Explanation

- Shared among all objects.
- Memory allocated once.
- Accessed using class name.
- Stores common data.
- Improves memory efficiency.

Example

```java
class Employee {
    static String company = "ABC";
}
```

Follow-up Questions

1. Where is static variable stored?
2. Why use static variables?
3. Can static variables be private?

Common Mistakes

- Expecting each object to have a separate copy.

Quick Revision

- Shared
- Single copy

- Class-level
- Memory efficient
- Access via class

---

33. What is a Static Method?

Question

Interviewer: "What is a Static Method?"

Answer

A static method belongs to the class and can be called without creating an object. It can directly access static members but cannot directly access non-static members.

Detailed Explanation

- Class-level method.
- No object required.
- Accesses static members directly.
- Cannot use `this`.
- Commonly used for utility methods.

Example

```java
class MathUtil {
    static int square(int n) {
        return n * n;
    }
}
```

Follow-up Questions

1. Why can't static methods use this?
2. Can static methods be overridden?
3. What is static binding?

Common Mistakes

- Accessing instance variables directly.
- Using this keyword.

Quick Revision

- Class-level
- No object
- Utility methods
- Cannot use this
- Static binding

---

34. What is a Static Block?

Question

Interviewer: "What is a Static Block in Java?"

Answer

A static block is used to initialize static variables when the class loads into memory. It executes only once regardless of how many objects are created.

Detailed Explanation

- Runs during class loading.
- Executes once.
- Initializes static data.
- Runs before constructor.
- Useful for configuration setup.

Example

```java
class Test {
    static {
        System.out.println("Static Block");
    }
}
```

Follow-up Questions

1. When does static block execute?
2. How many times does it execute?
3. Can there be multiple static blocks?

Common Mistakes

- Thinking it runs per object.
- Confusing constructor execution.

Quick Revision

- Executes once
- Class loading
- Static initialization
- Before constructor
- Multiple blocks allowed

---

35. What are Wrapper Classes?

Question

Interviewer: "What are Wrapper Classes?"

Answer

Wrapper Classes convert primitive data types into objects. They are useful when working with collections, generics, and utility methods that require objects.

Detailed Explanation

- Located in java.lang package.
- Wrap primitive values.
- Supports collections.
- Enables autoboxing.
- Enables unboxing.

Example

```java
Integer num = 10;
int value = num;
```

Follow-up Questions

1. What is autoboxing?
2. What is unboxing?
3. Why are wrapper classes needed?

Common Mistakes

- Confusing primitive and wrapper types.
- Ignoring null possibilities.

Quick Revision

- Primitive → Object
- Integer
- Double
- Autoboxing
- Unboxing

---

36. What is a Record Feature?

Question

Interviewer: "What is the Record feature in Java?"

Answer

Records are immutable data carrier classes that automatically generate constructors, getters, equals(), hashCode(), and toString() methods, reducing boilerplate code.

Detailed Explanation

- Immutable by default.
- Compact syntax.
- Auto-generated methods.
- Ideal for DTOs.

- Introduced in Java 17.

Example

```java
record Student(int id, String name) {}
```

Follow-up Questions

1. Are records immutable?
2. Can records extend classes?
3. Where are records useful?

Common Mistakes

- Treating records like mutable POJOs.

Quick Revision

- Immutable
- Compact syntax
- DTO usage
- Auto methods
- Java 17

---

37. What is a Marker Interface?

Question

Interviewer: "What is a Marker Interface?"

Answer

A Marker Interface is an interface with no methods or fields. It is used to provide metadata to the JVM or framework about a class.

Detailed Explanation

- Empty interface.
- Acts as a tag.

- Provides metadata.
- Used by frameworks.
- Influences runtime behavior.

Example

```java
public interface Serializable {
}
```

Follow-up Questions

1. Give examples of marker interfaces.
2. Why use marker interfaces?
3. Difference from annotations?

Common Mistakes

- Thinking marker interfaces contain methods.

Quick Revision

- Empty interface
- Metadata
- Serializable
- Cloneable
- Tagging mechanism

---

38. Difference between String, StringBuilder, and StringBuffer?

Question

Interviewer: "Compare String, StringBuilder, and StringBuffer."

Answer

String is immutable, while StringBuilder and StringBuffer are mutable. StringBuilder is faster but not thread-safe, whereas StringBuffer is thread-safe but slightly slower.

Detailed Explanation

String	StringBuilder	StringBuffer
Immutable	Mutable	Mutable
Thread-safe	No	Yes
Slow modifications	Fast	Moderate
Memory overhead	High	Low

Example

```java
StringBuilder sb = new StringBuilder();
sb.append("Java");
sb.append(" Spring");
```

Follow-up Questions

1. Which is fastest?
2. Which is thread-safe?
3. Why is String immutable?

Common Mistakes

- Saying StringBuilder is thread-safe.
- Using String in loops for concatenation.

Quick Revision

- String immutable
- Builder mutable
- Buffer thread-safe
- Builder faster
- Performance optimization

---

39. Difference between == and .equals()?

Question

Interviewer: "What is the difference between == and equals()?"

Answer

`==` compares memory references for objects, while `equals()` compares object contents. For primitive types, `==` compares values directly.

Detailed Explanation

- `==` checks reference equality.
- `equals()` checks content equality.
- String overrides equals().
- Important for object comparison.
- Used heavily in collections.

Example

```java
String s1 = new String("Java");
String s2 = new String("Java");

System.out.println(s1 == s2);      // false
System.out.println(s1.equals(s2)); // true
```

Follow-up Questions

1. Why does String override equals()?
2. How is equals related to hashCode()?
3. Can equals be overridden?

Common Mistakes

- Using == for String comparison.

Quick Revision

- == reference
- equals content
- String overrides equals
- Objects comparison
- Collections usage

40. What is a final Class?

Question

Interviewer: "What is a final class?"

Answer

A final class cannot be inherited. It is used when the class implementation should remain unchanged and secure.

Example

```java
final class Utility {
}
```

Quick Revision

- Cannot extend
- Secure implementation
- Immutable design support
- Used by String class
- Prevents inheritance

---

41. What is a final Method?

Question

Interviewer: "What is a final method?"

Answer

A final method cannot be overridden by child classes. It ensures that the method implementation remains unchanged.

Example

```java
class Parent {
    final void show() {}
```

```
}
```

Quick Revision

- Cannot override
- Inherited
- Preserves behavior
- Uses final keyword
- Security

---

42. What is a final Variable?

Question

Interviewer: "What is a final variable?"

Answer

A final variable can be assigned only once. After initialization, its value cannot be changed.

Example

```java
final int AGE = 25;
```

Quick Revision

- Constant value
- Assigned once
- Immutable reference
- Uses final
- Read-only

---

43. Explain the Thread Lifecycle.

Question

Interviewer: "Can you explain the lifecycle of a thread?"

Answer

A thread moves through multiple states such as New, Runnable, Running, Waiting/Blocked, and Terminated during execution.

Detailed Explanation

- New → Thread created.
- Runnable → Ready to run.
- Running → CPU assigned.
- Waiting/Blocked → Waiting for resource.
- Terminated → Execution complete.

Example

```java
Thread t = new Thread();
t.start();
```

Follow-up Questions

1. Difference between Runnable and Running?
2. What causes waiting state?
3. What is thread scheduling?

Common Mistakes

- Missing Runnable state.
- Confusing Waiting and Blocked.

Quick Revision

- New
- Runnable
- Running
- Waiting
- Terminated

---

44. What is Inter-Thread Communication?

Question

Interviewer: "What is Inter-Thread Communication?"

Answer

Inter-thread communication allows threads to coordinate their execution using methods like wait(), notify(), and notifyAll(). It helps avoid busy waiting and improves efficiency.

Detailed Explanation

- Thread coordination.
- Uses Object class methods.
- Requires synchronization.
- Prevents unnecessary CPU usage.
- Supports producer-consumer pattern.

Example

```java
wait();
notify();
```

Follow-up Questions

1. Why is synchronization required?
2. Producer-consumer example?
3. Difference between notify and notifyAll?

Common Mistakes

- Calling wait() outside synchronized block.

Quick Revision

- Thread coordination
- wait()
- notify()
- notifyAll()

- Synchronization

---

45. Difference between wait(), notify(), and notifyAll()?

Question

Interviewer: "Explain wait(), notify(), and notifyAll()."

Answer

`wait()` pauses a thread until notified. `notify()` wakes one waiting thread, while `notifyAll()` wakes all waiting threads on the same object monitor.

Example

```java
synchronized(obj) {
    obj.wait();
    obj.notify();
    obj.notifyAll();
}
```

Follow-up Questions

1. Why must they be called inside synchronized blocks?
2. Difference between sleep() and wait()?
3. When should notifyAll() be preferred?

Common Mistakes

- Using wait() without synchronization.
- Confusing sleep() with wait().

Quick Revision

- wait() pauses
- notify() one thread
- notifyAll() all threads
- Synchronization required
- Object class methods

---

46. How do you create a Custom Exception?

Question

Interviewer: "How do you create a custom exception in Java?"

Answer

A custom exception is created by extending either Exception or RuntimeException. It is used to represent application-specific error conditions with meaningful messages.

Detailed Explanation

- Extend Exception (checked).
- Extend RuntimeException (unchecked).
- Add constructors.
- Provide meaningful messages.
- Improve error handling.

Example

```java
class InvalidAgeException extends Exception {

    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Usage:

```java
if(age < 18) {
    throw new InvalidAgeException("Age must be 18+");
}
```

Follow-up Questions

1. Difference between checked and unchecked exceptions?
2. Why create custom exceptions?
3. When should RuntimeException be used?

Common Mistakes

- Extending Throwable directly.
- Not providing meaningful exception messages.

Quick Revision

- Extend Exception
- Extend RuntimeException
- Custom business errors
- throw keyword
- Better readability

Next batch:

Part 3 (Q47-Q58) — Design Patterns & SOLID Principles

This section is extremely important because interviewers frequently ask:

- Singleton Pattern
- Factory Pattern
- Builder Pattern
- Strategy Pattern
- SRP, OCP, LSP, ISP, DIP

and often expect both theory and real-world examples.

2026-06-22 22:11:03 **You Asked**

Part 3 (Q47-Q58) — Design Patterns & SOLID Principles

ChatGPT gpt-5-5

47. What are Design Patterns?

Question

Interviewer: "What are Design Patterns and why are they important?"

Answer

Design Patterns are proven solutions to common software design problems. They provide reusable templates for building maintainable, scalable, and flexible applications. They are not ready-made code but best practices for solving recurring design challenges.

Detailed Explanation

- Reusable design solutions.
- Improve code maintainability.
- Reduce code duplication.
- Promote best practices.
- Make systems easier to scale.
- Improve communication among developers.
- Categorized as Creational, Structural, and Behavioral.

Example

A Singleton Pattern ensures only one database connection manager exists throughout the application.

Follow-up Questions

1. What are the types of Design Patterns?
2. Which design patterns have you used?
3. Is Singleton a Creational Pattern?

Common Mistakes

- Saying design patterns are frameworks.
- Memorizing definitions without understanding use cases.

Quick Revision

- Reusable solutions
- Best practices
- Maintainable code
- Scalable systems
- Creational/Structural/Behavioral

48. What is the Singleton Design Pattern?

Question

Interviewer: "Explain the Singleton Design Pattern."

Answer

Singleton ensures that only one instance of a class exists throughout the application and provides a global access point to that instance. It is commonly used for logging, configuration, and database connection management.

Detailed Explanation

- Creates only one object.
- Global access point.
- Saves memory.
- Controls shared resources.
- Uses private constructor.
- Uses static instance variable.
- Common in Spring singleton beans.

Example

```java
public class Singleton {

    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if(instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Follow-up Questions

1. Why is constructor private?

2. Is Singleton thread-safe?
3. Where have you used Singleton?

Common Mistakes

- Forgetting private constructor.
- Ignoring thread safety concerns.

Quick Revision

- One instance
- Private constructor
- Static instance
- Global access
- Creational pattern

---

49. What is the Factory Design Pattern?

Question

Interviewer: "What is the Factory Design Pattern?"

Answer

Factory Pattern provides an interface for creating objects without exposing the object creation logic to the client. The client requests an object, and the factory decides which object to create.

Detailed Explanation

- Creational pattern.
- Hides object creation.
- Promotes loose coupling.
- Centralizes creation logic.
- Easier maintenance.
- Simplifies object management.

Example

```
java
```

```
interface Vehicle {}

class Car implements Vehicle {}

class Bike implements Vehicle {}

class VehicleFactory {

    public Vehicle getVehicle(String type) {
        if(type.equals("CAR"))
            return new Car();

        return new Bike();
    }
}
```

Follow-up Questions

1. Why use Factory Pattern?
2. Difference between Factory and Builder?
3. Have you used Factory in Spring?

Common Mistakes

- Confusing Factory with Singleton.
- Creating objects directly in client code.

Quick Revision

- Object creation
- Loose coupling
- Centralized logic
- Creational pattern
- Better maintainability

---

50. What is the Builder Design Pattern?

Question

Interviewer: "What is the Builder Pattern and when would you use it?"

Answer

Builder Pattern is used to create complex objects step by step. It is especially useful when an object has many optional parameters, making code more readable and maintainable.

Detailed Explanation

- Creational pattern.
- Builds objects incrementally.
- Handles optional parameters.
- Improves readability.
- Avoids large constructors.
- Creates immutable objects easily.

Example

```java
Employee emp = Employee.builder()
        .id(1)
        .name("John")
        .department("IT")
        .build();
```

Follow-up Questions

1. Why is Builder better than multiple constructors?
2. How does Lombok support Builder Pattern?
3. Is Builder immutable?

Common Mistakes

- Using constructors with many parameters.
- Confusing Builder with Factory.

Quick Revision

- Complex object creation
- Step-by-step building
- Optional fields

- Readability
- Lombok @Builder

---

51. What is the Strategy Design Pattern?

Question

Interviewer: "Can you explain the Strategy Pattern?"

Answer

Strategy Pattern allows multiple algorithms or behaviors to be encapsulated separately and selected at runtime. It helps eliminate large if-else blocks and promotes flexibility.

Detailed Explanation

- Behavioral pattern.
- Encapsulates algorithms.
- Runtime behavior selection.
- Reduces conditional logic.
- Promotes Open/Closed Principle.
- Easy to add new strategies.

Example

```java
interface PaymentStrategy {
    void pay();
}

class CreditCardPayment implements PaymentStrategy {
    public void pay() {
        System.out.println("Paid using card");
    }
}

class UpiPayment implements PaymentStrategy {
    public void pay() {
        System.out.println("Paid using UPI");
    }
}
```

Follow-up Questions

1. Where is Strategy Pattern used?
2. Difference between Strategy and Factory?
3. What problem does Strategy solve?

Common Mistakes

- Using if-else chains instead of strategies.
- Confusing strategy selection with object creation.

Quick Revision

- Behavioral pattern
- Multiple algorithms
- Runtime selection
- Flexible design
- Reduces if-else

---

52. Why is the constructor private in Singleton?

Question

Interviewer: "Why do we make the constructor private in Singleton?"

Answer

The constructor is made private to prevent external classes from creating multiple objects using the `new` keyword. This ensures that only one instance of the class exists.

Detailed Explanation

- Prevents object creation.
- Enforces Singleton rule.
- Controls instantiation.
- Ensures single instance.
- Access only through factory method.

Example

```java
private Singleton() {
}
```

Follow-up Questions

1. What happens if constructor is public?
2. How is Singleton accessed?
3. Can reflection break Singleton?

Common Mistakes

- Forgetting private constructor.
- Not understanding object creation control.

Quick Revision

- Prevents new keyword
- Single instance
- Object control
- Singleton requirement
- Private constructor

---

53. Why is the Singleton instance variable static and private?

Question

Interviewer: "Why is the Singleton instance variable static and private?"

Answer

The instance variable is static because it belongs to the class and should be shared globally. It is private to prevent direct access and enforce controlled access through a public method.

Detailed Explanation

- Static → one shared instance.
- Private → encapsulation.
- Accessible through class method.
- Supports global access.

- Prevents unauthorized modification.

Example

```java
private static Singleton instance;
```

Follow-up Questions

1. Why can't it be public?
2. Why must it be static?
3. How is thread safety handled?

Common Mistakes

- Making instance variable public.
- Using non-static instance variable.

Quick Revision

- Static shared object
- Private access
- Encapsulation
- Global availability
- Singleton design

---

54. What is the Single Responsibility Principle (SRP)?

Question

Interviewer: "Explain the Single Responsibility Principle."

Answer

SRP states that a class should have only one reason to change. A class should focus on a single responsibility or functionality, making the code easier to maintain and test.

Detailed Explanation

- First SOLID principle.

- One responsibility per class.
- Easier maintenance.
- Better readability.
- Easier testing.
- Reduces side effects.

Example

❌ Bad

```java
class Employee {
    void saveEmployee() {}
    void generateReport() {}
}
```

✅ Good

```java
class EmployeeService {}
class ReportService {}
```

Follow-up Questions

1. Why is SRP important?
2. How does SRP improve testing?
3. Example from your project?

Common Mistakes

- Putting business logic and database logic in one class.
- Creating God Classes.

Quick Revision

- One responsibility
- One reason to change
- Easier testing
- Better maintenance
- SOLID principle

55. What is the Open/Closed Principle (OCP)?

Question

Interviewer: "What is the Open/Closed Principle?"

Answer

OCP states that software entities should be open for extension but closed for modification. New functionality should be added without changing existing code.

Detailed Explanation

- Second SOLID principle.
- Extend behavior without modifying.
- Reduces risk of bugs.
- Supports scalability.
- Often implemented using interfaces.
- Encourages polymorphism.

Example

```java
interface Payment {
    void pay();
}
```

Adding a new payment type requires a new implementation, not changing existing code.

Follow-up Questions

1. How does OCP relate to interfaces?
2. How does Strategy Pattern support OCP?
3. Example in Spring Boot?

Common Mistakes

- Modifying existing classes repeatedly.
- Using large if-else structures.

Quick Revision

- Open for extension
- Closed for modification
- Interfaces
- Polymorphism
- Scalable code

---

56. What is the Liskov Substitution Principle (LSP)?

Question

Interviewer: "Explain the Liskov Substitution Principle."

Answer

LSP states that child classes should be able to replace parent classes without affecting program correctness. Subclasses must honor the behavior expected from the parent class.

Detailed Explanation

- Third SOLID principle.
- Supports proper inheritance.
- Child must not break parent behavior.
- Improves reliability.
- Ensures substitutability.
- Promotes robust design.

Example

❌ Bad Example

```java
class Bird {
    void fly() {}
}

class Penguin extends Bird {
    void fly() {
        throw new UnsupportedOperationException();
    }
```

```
}
```

Follow-up Questions

1. Why is Penguin a common LSP example?
2. How does LSP affect inheritance?
3. What happens if LSP is violated?

Common Mistakes

- Using inheritance where it doesn't fit.
- Creating subclasses that break behavior.

Quick Revision

- Proper inheritance
- Substitutability
- Child behaves like parent
- Reliable code
- SOLID principle

---

57. What is the Interface Segregation Principle (ISP)?

Question

Interviewer: "What is Interface Segregation Principle?"

Answer

ISP states that clients should not be forced to depend on methods they do not use. Large interfaces should be split into smaller, specific interfaces.

Detailed Explanation

- Fourth SOLID principle.
- Small focused interfaces.
- Avoid unnecessary methods.
- Improves maintainability.
- Reduces coupling.
- Easier implementation.

Example

❌ Bad

```java
interface Worker {
    void work();
    void eat();
}
```

✅ Better

```java
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}
```

Follow-up Questions

1. Why avoid large interfaces?
2. How does ISP reduce coupling?
3. Real-world example?

Common Mistakes

- Creating huge interfaces.
- Forcing classes to implement unused methods.

Quick Revision

- Small interfaces
- Focused contracts
- Reduced coupling
- Easier maintenance
- SOLID principle

---

58. What is the Dependency Inversion Principle (DIP)?

Question

Interviewer: "Explain Dependency Inversion Principle."

Answer

DIP states that high-level modules should not depend on low-level modules; both should depend on abstractions. This principle promotes loose coupling and is the foundation of Dependency Injection in Spring.

Detailed Explanation

- Fifth SOLID principle.
- Depend on interfaces.
- Loose coupling.
- Easier testing.
- Flexible implementations.
- Core concept behind Spring DI.

Example

❌ Tight Coupling

```
java

class Service {
    private MySQLDatabase db = new MySQLDatabase();
}
```

✅ DIP

```
java

interface Database {}

class Service {
    private Database db;
}
```

Follow-up Questions

1. How does Spring implement DIP?

2. Difference between DIP and DI?
3. Why use interfaces?

Common Mistakes

- Confusing DIP with Dependency Injection.
- Depending directly on concrete classes.

Quick Revision

- Depend on abstractions
- Loose coupling
- Easier testing
- Foundation of Spring DI
- SOLID principle

Most Important Questions From Part 3

If you have only 15 minutes before an interview, focus on:

1. Singleton Pattern
2. Factory Pattern
3. Builder Pattern
4. Strategy Pattern
5. SRP
6. OCP
7. DIP
8. Abstract Class vs Interface
9. Overloading vs Overriding
10. Dependency Injection vs Dependency Inversion

Next: **Part 4 (Q59–Q99) — Spring Framework, Spring Boot, Spring MVC, Bean Lifecycle, Exception Handling, DTOs, and Lombok**. This is typically the highest-weightage section for Java Spring Boot fresher interviews.

2026-06-22 22:12:20 **You Asked**

Next

ChatGPT gpt-5-5

Part 4 (Q59–Q99) — Spring Framework & Spring Boot

59. What are the features of Spring Framework?

Question

Interviewer: "What are the main features of Spring Framework?"

Answer

Spring Framework is a lightweight, modular framework for building Java applications. Its key features include Dependency Injection (DI), Inversion of Control (IoC), Aspect-Oriented Programming (AOP), transaction management, Spring MVC, and easy integration with databases and third-party libraries.

Detailed Explanation

- Lightweight and modular architecture.
- Supports Dependency Injection.
- Implements IoC Container.
- Provides AOP support.
- Built-in transaction management.
- Spring MVC for web applications.
- Easy integration with JPA, Hibernate, and databases.

Example

```java
@Service
public class UserService {
}
```

Spring automatically manages the object lifecycle.

Follow-up Questions

1. What is IoC?
2. What is Dependency Injection?
3. What modules are available in Spring?

Common Mistakes

- Saying Spring and Spring Boot are the same.
- Not mentioning DI and IoC.

Quick Revision

- DI
- IoC
- AOP
- Transaction Management
- Spring MVC

---

60. Difference between Spring and Spring Boot?

Question

Interviewer: "What is the difference between Spring and Spring Boot?"

Answer

Spring is a framework for building enterprise applications, while Spring Boot is an extension of Spring that simplifies configuration and development using auto-configuration and embedded servers.

Detailed Explanation

Spring	Spring Boot
Requires manual configuration	Auto-configuration
External server needed	Embedded server
More setup	Faster development
XML-heavy (traditionally)	Annotation-based
More boilerplate	Less boilerplate

Example

Spring Boot:

```java
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class,args);
    }
}
```

Follow-up Questions

1. What is auto-configuration?
2. What embedded servers are supported?
3. Why use Spring Boot?

Common Mistakes

- Saying Spring Boot replaces Spring.
- Ignoring embedded server feature.

Quick Revision

- Spring Boot = Spring + Automation
- Embedded Tomcat
- Auto Configuration
- Faster setup
- Less XML

---

61. What is Dependency Injection?

Question

Interviewer: "What is Dependency Injection?"

Answer

Dependency Injection is a design pattern where the Spring container provides required dependencies to an object instead of the object creating them itself. This promotes loose coupling and easier

testing.

Detailed Explanation

- Core Spring concept.
- Dependencies injected by container.
- Promotes loose coupling.
- Easier unit testing.
- Supports constructor injection.
- Supports setter injection.

Example

```java
@Service
public class UserService {

    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }
}
```

Follow-up Questions

1. Types of DI?
2. Constructor vs Setter Injection?
3. Benefits of DI?

Common Mistakes

- Confusing DI with IoC.
- Creating objects using `new`.

Quick Revision

- Loose coupling
- Constructor Injection
- Setter Injection

- Easier testing
- Spring managed

---

62. What is Inversion of Control (IoC)?

Question

Interviewer: "What is IoC in Spring?"

Answer

IoC means transferring control of object creation and lifecycle management from the application code to the Spring container. Spring manages beans and their dependencies automatically.

Detailed Explanation

- Foundation of Spring.
- Container controls object creation.
- Manages lifecycle.
- Supports Dependency Injection.
- Reduces tight coupling.

Example

Instead of:

```java
UserService service = new UserService();
```

Spring creates the object automatically.

Follow-up Questions

1. Difference between IoC and DI?
2. What is IoC Container?
3. Which container manages beans?

Common Mistakes

- Saying IoC and DI are identical.

Quick Revision

- Container control
- Bean management
- Object lifecycle
- DI implementation
- Loose coupling

---

63. What is a Spring Bean?

Question

Interviewer: "What is a Spring Bean?"

Answer

A Spring Bean is an object managed by the Spring IoC container. Spring creates, configures, and manages the lifecycle of beans automatically.

Detailed Explanation

- Managed by Spring.
- Stored in IoC container.
- Lifecycle controlled by Spring.
- Can be injected into other beans.
- Created using annotations or configuration.

Example

```java
@Service
public class EmployeeService {
}
```

Follow-up Questions

1. How are beans created?
2. Where are beans stored?
3. What is bean scope?

Common Mistakes

- Thinking every Java object is a Spring Bean.

Quick Revision

- Managed object
- IoC Container
- Dependency Injection
- Lifecycle managed
- Reusable component

---

64. What are Bean Scopes in Spring?

Question

Interviewer: "What bean scopes are available in Spring?"

Answer

Bean scope defines the lifecycle and visibility of a bean. Common scopes are Singleton, Prototype, Request, Session, Application, and WebSocket.

Detailed Explanation

- Singleton → one bean per container.
- Prototype → new bean each request.
- Request → one bean per HTTP request.
- Session → one bean per session.
- Application → one bean per servlet context.
- WebSocket → one bean per WebSocket session.

Example

```java
@Scope("prototype")
@Component
public class Employee {
}
```

Follow-up Questions

1. Default scope?
2. Difference between Singleton and Prototype?
3. Which scope is web-specific?

Common Mistakes

- Assuming Prototype is default.

Quick Revision

- Singleton
- Prototype
- Request
- Session
- Application

---

65. What is Singleton Scope?

Question

Interviewer: "What is Singleton Scope in Spring?"

Answer

Singleton is the default bean scope in Spring. Only one instance of the bean is created and shared throughout the application context.

Detailed Explanation

- Default scope.
- One bean instance.
- Shared across application.
- Memory efficient.
- Managed by Spring container.

Example

```java
@Service
public class UserService {
```

```
}
```

Follow-up Questions

1. Is Singleton thread-safe?
2. Why is Singleton default?
3. How many objects are created?

Common Mistakes

- Assuming Singleton means JVM-wide singleton.

Quick Revision

- Default scope
- One bean
- Shared instance
- Memory efficient
- Container managed

---

66. What is Prototype Scope?

Question

Interviewer: "What is Prototype Scope?"

Answer

Prototype scope creates a new bean instance every time it is requested from the container.

Detailed Explanation

- New object every request.
- Not shared.
- Useful for stateful beans.
- More memory usage.
- Container manages creation only.

Example

```
java
```

```
@Scope("prototype")
@Component
public class Employee {
}
```

Follow-up Questions

1. When should Prototype be used?
2. Is destruction managed?
3. Difference from Singleton?

Common Mistakes

- Assuming Spring manages destruction.

Quick Revision

- New object
- Not shared
- Stateful beans
- More memory
- Prototype scope

---

67. What is Request Scope?

Question

Interviewer: "What is Request Scope?"

Answer

Request scope creates one bean instance per HTTP request. Once the request is completed, the bean is destroyed.

Detailed Explanation

- Web applications only.
- One bean per request.
- Destroyed after response.
- Request-specific data.

- Not shared.

Example

```java
@Scope(value="request")
@Component
public class RequestBean {
}
```

Follow-up Questions

1. Is it available outside web apps?
2. Difference from Session Scope?
3. Lifecycle duration?

Common Mistakes

- Using request scope in non-web applications.

Quick Revision

- One request
- One bean
- Temporary lifecycle
- Web scope
- Request-specific

---

68. What is Session Scope?

Question

Interviewer: "What is Session Scope?"

Answer

Session scope creates one bean per user session. The bean remains available until the session expires or is invalidated.

Detailed Explanation

- One bean per session.
- User-specific data.
- Available across requests.
- Destroyed when session ends.
- Web applications only.

Example

```java
@Scope("session")
@Component
public class UserSession {
}
```

Follow-up Questions

1. Difference from Request Scope?
2. When is bean destroyed?
3. Typical use cases?

Common Mistakes

- Storing large objects in session.

Quick Revision

- Per user session
- Shared across requests
- User-specific
- Web application
- Session ends

---

69. What is Application Scope?

Question

Interviewer: "What is Application Scope?"

Answer

Application scope creates one bean per ServletContext and shares it across the entire web application.

Detailed Explanation

- One bean per application.
- Shared globally.
- Available to all users.
- Lives until application shutdown.
- Web-specific scope.

Example

```java
@Scope("application")
@Component
public class GlobalConfig {
}
```

Follow-up Questions

1. Difference from Singleton?
2. Who shares the bean?
3. Lifecycle duration?

Common Mistakes

- Confusing Singleton and Application scope.

Quick Revision

- Global bean
- Shared application-wide
- ServletContext
- Long lifecycle
- Web scope

---

70. What is WebSocket Scope?

Question

Interviewer: "What is WebSocket Scope?"

Answer

WebSocket scope creates one bean instance for each WebSocket session and maintains it until the connection closes.

Detailed Explanation

- One bean per WebSocket session.
- Supports real-time communication.
- Session-specific state.
- Useful in chat applications.
- Ends when socket disconnects.

Example

```java
@Scope("websocket")
@Component
public class ChatSession {
}
```

Follow-up Questions

1. Where is WebSocket used?
2. Difference from Session Scope?
3. When is bean destroyed?

Common Mistakes

- Using WebSocket scope without WebSocket support.

Quick Revision

- WebSocket session
- Real-time apps
- One bean per connection
- Chat systems

- Connection lifecycle

These first 12 questions (59–70) cover:

- Spring Basics
- Spring vs Spring Boot
- DI & IoC
- Spring Beans
- All Bean Scopes

These are among the most frequently asked Spring Boot fresher interview questions.

Next: **Q71–Q85** (Bean Lifecycle, Spring Annotations, Spring MVC Architecture, DispatcherServlet, Request Flow). This is another very high-frequency interview section.

2026-06-22 22:13:18 **You Asked**

Next

ChatGPT gpt-5-5

Part 4 (Q71–Q85) — Bean Lifecycle, Spring Annotations & Spring MVC

---

71. Explain the Bean Lifecycle in Spring.

Question

Interviewer: "Can you explain the lifecycle of a Spring Bean?"

Answer

The Spring Bean Lifecycle starts when the container creates the bean, injects dependencies, initializes it, makes it available for use, and finally destroys it when the application context is closed.

Detailed Explanation

- Spring container creates the bean.
- Dependencies are injected.

- Bean-aware interfaces are called.
- Initialization methods execute.
- Bean is ready for use.
- Destruction methods execute before removal.

Lifecycle Flow:

```text
Bean Creation
      ↓
Dependency Injection
      ↓
@PostConstruct
      ↓
Bean Ready
      ↓
Application Running
      ↓
@PreDestroy
      ↓
Bean Destroyed
```

Example

```java
@Component
public class EmployeeService {

    @PostConstruct
    public void init() {
        System.out.println("Bean Initialized");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("Bean Destroyed");
    }
}
```

Follow-up Questions

1. What is @PostConstruct?
2. What is @PreDestroy?

3. Does Spring destroy Prototype beans?

Common Mistakes

- Forgetting dependency injection step.
- Saying Spring manages Prototype bean destruction.

Quick Revision

- Bean creation
- Dependency injection
- Initialization
- Usage
- Destruction

---

72. What happens during Bean Initialization?

Question

Interviewer: "What happens during Bean Initialization?"

Answer

After creating the bean and injecting dependencies, Spring executes initialization callbacks such as `@PostConstruct`, `InitializingBean.afterPropertiesSet()`, or custom init methods.

Detailed Explanation

- Bean object created.
- Dependencies injected.
- Initialization callback executed.
- Resource setup happens.
- Bean becomes ready for use.

Example

```java
@PostConstruct
public void init() {
    System.out.println("Initializing...");
```

```
}
```

Follow-up Questions

1. What is @PostConstruct?
2. When is afterPropertiesSet() called?
3. Can multiple init methods exist?

Common Mistakes

- Assuming initialization happens before dependency injection.

Quick Revision

- After DI
- @PostConstruct
- Initialization logic
- Resource setup
- Bean ready

---

73. What happens during Bean Destruction?

Question

Interviewer: "What happens during Bean Destruction?"

Answer

When the application context closes, Spring executes destruction callbacks such as `@PreDestroy` or custom destroy methods to release resources and perform cleanup operations.

Detailed Explanation

- Triggered during shutdown.
- Releases resources.
- Closes connections.
- Executes cleanup code.
- Mainly for Singleton beans.

Example

```java
@PreDestroy
public void cleanup() {
    System.out.println("Cleaning resources");
}
```

Follow-up Questions

1. What is @PreDestroy?
2. Does Prototype bean support destruction?
3. Why is cleanup important?

Common Mistakes

- Assuming destruction happens for every scope.

Quick Revision

- Cleanup
- @PreDestroy
- Resource release
- Application shutdown
- Singleton scope

---

74. What is @SpringBootApplication?

Question

Interviewer: "What does @SpringBootApplication do?"

Answer

`@SpringBootApplication` is the main annotation of a Spring Boot application. It combines `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan` into a single annotation.

Detailed Explanation

- Entry point annotation.
- Enables auto-configuration.
- Enables component scanning.

- Supports bean configuration.
- Reduces boilerplate code.

Example

```java
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class,args);
    }
}
```

Follow-up Questions

1. What annotations does it contain?
2. What is component scanning?
3. What is auto-configuration?

Common Mistakes

- Not knowing the three annotations it combines.

Quick Revision

- Main annotation
- Auto Configuration
- Component Scan
- Configuration
- Spring Boot startup

---

75. What is @Component?

Question

Interviewer: "What is @Component?"

Answer

`@Component` marks a class as a Spring-managed bean. During component scanning, Spring automatically detects and registers it in the IoC container.

Detailed Explanation

- Generic stereotype annotation.
- Creates Spring bean.
- Auto-detected by scanning.
- Managed by container.
- Parent annotation for others.

Example

```java
@Component
public class UtilityService {
}
```

Follow-up Questions

1. Difference from @Service?
2. Difference from @Repository?
3. How is it discovered?

Common Mistakes

- Using @Component everywhere instead of specialized annotations.

Quick Revision

- Generic bean
- Component scan
- Spring managed
- IoC container
- Base stereotype

---

76. What is @Service?

Question

Interviewer: "What is @Service?"

Answer

`@Service` is a specialized form of `@Component` used for business logic classes. It improves readability and clearly identifies the service layer.

Detailed Explanation

- Business layer annotation.
- Specialized @Component.
- Spring-managed bean.
- Improves architecture clarity.
- Used in service classes.

Example

```java
@Service
public class EmployeeService {
}
```

Follow-up Questions

1. Difference from @Component?
2. Why use @Service?
3. Can Spring create a bean from it?

Common Mistakes

- Putting repository logic inside service classes.

Quick Revision

- Business logic
- Service layer
- Spring bean
- Specialized component

- Readable architecture

---

77. What is @Repository?

Question

Interviewer: "What is @Repository?"

Answer

`@Repository` is used for data access layer classes. It indicates that the class interacts with the database and enables Spring's exception translation mechanism.

Detailed Explanation

- DAO layer annotation.
- Specialized @Component.
- Database operations.
- Exception translation support.
- Managed by Spring.

Example

```java
@Repository
public class EmployeeRepository {
}
```

Follow-up Questions

1. Difference from @Service?
2. What is exception translation?
3. Can repositories be injected?

Common Mistakes

- Writing business logic inside repositories.

Quick Revision

- Database layer

- DAO
- Exception translation
- Spring bean
- Data access

---

78. What is @Controller?

Question

Interviewer: "What is @Controller?"

Answer

`@Controller` is used in Spring MVC to handle web requests and return view names such as JSP, Thymeleaf, or HTML pages.

Detailed Explanation

- MVC controller.
- Handles HTTP requests.
- Returns views.
- Used in web applications.
- Managed by Spring.

Example

```java
@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "index";
    }
}
```

Follow-up Questions

1. Difference from @RestController?
2. What does it return?

3. How are requests mapped?

Common Mistakes

- Using @Controller for REST APIs.

Quick Revision

- MVC layer
- Handles requests
- Returns views
- Web pages
- Spring MVC

---

79. What is @RestController?

Question

Interviewer: "What is @RestController?"

Answer

`@RestController` is used to build REST APIs. It combines `@Controller` and `@ResponseBody`, allowing methods to return JSON or XML directly.

Detailed Explanation

- REST API annotation.
- Returns JSON/XML.
- Combines Controller + ResponseBody.
- No view resolution.
- Common in Spring Boot APIs.

Example

```java

@RestController
public class EmployeeController {

    @GetMapping("/hello")
    public String hello() {
```

```
        return "Hello";
    }
}
```

Follow-up Questions

1. Difference from @Controller?
2. What is @ResponseBody?
3. Can it return objects?

Common Mistakes

- Returning view names from RestController.

Quick Revision

- REST APIs
- JSON response
- ResponseBody included
- No views
- Spring Boot APIs

---

80. What is @Autowired?

Question

Interviewer: "What is @Autowired?"

Answer

`@Autowired` is used by Spring to automatically inject dependencies into a bean. It reduces manual object creation and supports Dependency Injection.

Detailed Explanation

- Dependency Injection annotation.
- Automatic bean wiring.
- Supports constructor injection.
- Supports field injection.
- Supports setter injection.

Example

```java
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;
}
```

Follow-up Questions

1. Constructor vs Field Injection?
2. Is @Autowired mandatory?
3. What happens if multiple beans exist?

Common Mistakes

- Overusing field injection.
- Not understanding constructor injection.

Quick Revision

- Dependency Injection
- Auto wiring
- Bean injection
- Constructor preferred
- Loose coupling

---

81. What is @Configuration?

Question

Interviewer: "What is @Configuration?"

Answer

`@Configuration` indicates that a class contains bean definitions. Spring processes it and registers the returned objects as beans.

Detailed Explanation

- Configuration class.
- Defines beans.
- Processed by Spring.
- Alternative to XML configuration.
- Supports Java-based configuration.

Example

```java
@Configuration
public class AppConfig {
}
```

Follow-up Questions

1. Difference from Component?
2. Why use Configuration?
3. What is Java Config?

Common Mistakes

- Using Configuration without bean definitions.

Quick Revision

- Java Config
- Bean definitions
- Alternative to XML
- Spring managed
- Configuration class

---

82. What is @Bean?

Question

Interviewer: "What is @Bean?"

Answer

`@Bean` is used inside a `@Configuration` class to explicitly create and register objects as Spring beans.

Detailed Explanation

- Creates Spring bean.
- Used inside Configuration.
- Manual bean registration.
- Useful for third-party classes.
- Managed by Spring.

Example

```java
@Configuration
public class Config {

    @Bean
    public EmployeeService employeeService() {
        return new EmployeeService();
    }
}
```

Follow-up Questions

1. Difference from Component?
2. Why use Bean annotation?
3. Where can Bean be used?

Common Mistakes

- Using @Bean outside Configuration classes.

Quick Revision

- Bean creation
- Java Config
- Spring managed
- Explicit registration

- Third-party classes

---

83. What is @Value?

Question

Interviewer: "What is @Value used for?"

Answer

`@Value` injects values from properties files, environment variables, or expressions into Spring beans.

Detailed Explanation

- Reads configuration values.
- Injects properties.
- Supports expressions.
- Externalizes configuration.
- Useful for application.properties.

Example

```java
@Value("${server.port}")
private int port;
```

Follow-up Questions

1. From where are values loaded?
2. Can it read environment variables?
3. Difference from ConfigurationProperties?

Common Mistakes

- Hardcoding values instead of using properties.

Quick Revision

- Property injection
- External configuration

- application.properties
- Environment variables
- Flexible configuration

---

84. Difference between @Controller and @RestController?

Question

Interviewer: "Compare @Controller and @RestController."

Answer

`@Controller` returns views like JSP or Thymeleaf pages, while `@RestController` returns data such as JSON or XML directly to the client.

Detailed Explanation

@Controller	@RestController
Returns views	Returns JSON/XML
MVC applications	REST APIs
Uses View Resolver	No View Resolver
Needs @ResponseBody	Included automatically

Example

```java
@Controller
public class HomeController {}
```

```java
@RestController
public class ApiController {}
```

Follow-up Questions

1. What is @ResponseBody?
2. Which annotation is used for REST APIs?

3. Can Controller return JSON?

Common Mistakes

- Using Controller for REST APIs.

Quick Revision

- Controller → View
- RestController → JSON
- ResponseBody
- MVC
- REST

---

85. Explain Spring MVC Architecture.

Question

Interviewer: "Can you explain Spring MVC Architecture?"

Answer

Spring MVC follows the Model-View-Controller pattern. The DispatcherServlet receives requests, forwards them to controllers, processes business logic through services, interacts with repositories, and returns responses to the client.

Detailed Explanation

```text
Client
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

- DispatcherServlet acts as Front Controller.

- Controller handles requests.
- Service contains business logic.
- Repository accesses database.
- Response returned to client.

Example

```java
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable Long id) {
        return service.getEmployee(id);
    }
}
```

Follow-up Questions

1. What is DispatcherServlet?
2. Explain Request Flow.
3. What is the role of Controller?

Common Mistakes

- Not mentioning DispatcherServlet.
- Mixing Service and Repository responsibilities.

Quick Revision

- MVC architecture
- DispatcherServlet
- Controller
- Service
- Repository

High-Probability Interview Questions from Q71–Q85

1. Bean Lifecycle ⭐⭐⭐⭐⭐
2. @SpringBootApplication ⭐⭐⭐⭐⭐
3. @Autowired ⭐⭐⭐⭐⭐
4. @Component vs @Service vs @Repository ⭐⭐⭐⭐⭐
5. @Controller vs @RestController ⭐⭐⭐⭐⭐
6. Spring MVC Architecture ⭐⭐⭐⭐⭐
7. @Bean vs @Component ⭐⭐⭐⭐
8. Bean Scopes ⭐⭐⭐⭐

Next section: **Q86–Q99** (DispatcherServlet, Request Flow, Global Exception Handling, DTOs, Lombok annotations). These are extremely common in Spring Boot project discussions.

2026-06-22 22:14:28 **You Asked**

Next

ChatGPT gpt-5-5

Part 4 (Q86–Q99) — DispatcherServlet, Exception Handling, DTOs & Lombok

---

86. What is DispatcherServlet?

Question

Interviewer: "What is DispatcherServlet in Spring MVC?"

Answer

DispatcherServlet is the Front Controller of Spring MVC. It receives all incoming HTTP requests, routes them to the appropriate controller, and returns the response to the client.

Detailed Explanation

- Central controller in Spring MVC.
- Handles all incoming requests.
- Maps requests to controllers.

- Coordinates with Handler Mapping.
- Processes responses.
- Improves request management.

Example

```text
Client Request
      ↓
DispatcherServlet
      ↓
Controller
      ↓
Service
      ↓
Response
```

Follow-up Questions

1. Why is it called Front Controller?
2. What happens after DispatcherServlet receives a request?
3. Does every request pass through DispatcherServlet?

Common Mistakes

- Confusing DispatcherServlet with Controller.
- Not knowing it is the entry point of Spring MVC.

Quick Revision

- Front Controller
- Receives requests
- Routes requests
- Returns responses
- Spring MVC entry point

---

87. Explain Spring MVC Request Flow.

Question

Interviewer: "Can you explain the Spring MVC request flow?"

Answer

When a client sends a request, DispatcherServlet receives it and forwards it to the appropriate controller. The controller calls the service layer, which interacts with the repository and database. The result is returned back through the same layers to the client.

Detailed Explanation

```text
Client
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

- Client sends request.
- DispatcherServlet receives it.
- Controller handles request.
- Service executes business logic.
- Repository accesses database.
- Response returned to client.

Example

```java
@GetMapping("/{id}")
public Employee getEmployee(@PathVariable Long id){
    return service.getEmployee(id);
}
```

Follow-up Questions

1. Where does business logic reside?

2. What is the role of Repository?
3. Why separate layers?

Common Mistakes

- Putting business logic inside Controller.
- Mixing Service and Repository responsibilities.

Quick Revision

- DispatcherServlet
- Controller
- Service
- Repository
- Database

---

88. What is Global Exception Handling?

Question

Interviewer: "What is Global Exception Handling in Spring Boot?"

Answer

Global Exception Handling is a mechanism to handle exceptions centrally across the application instead of writing try-catch blocks in every controller. It improves maintainability and provides consistent error responses.

Detailed Explanation

- Centralized exception handling.
- Reduces duplicate code.
- Improves API consistency.
- Uses @ControllerAdvice.
- Uses @ExceptionHandler.
- Better user experience.

Example

```
java
```

```
@ControllerAdvice
public class GlobalExceptionHandler {
}
```

Follow-up Questions

1. Why not use try-catch everywhere?
2. What annotation is used?
3. How are custom exceptions handled?

Common Mistakes

- Using try-catch in every controller method.
- Not returning proper error responses.

Quick Revision

- Centralized handling
- @ControllerAdvice
- @ExceptionHandler
- Cleaner code
- Consistent responses

---

89. What is @ControllerAdvice?

Question

Interviewer: "What is @ControllerAdvice?"

Answer

@ControllerAdvice is a Spring annotation used to handle exceptions globally across multiple controllers. It allows centralized error handling logic.

Detailed Explanation

- Global exception handler.
- Applies to all controllers.
- Centralized logic.
- Reduces duplication.

- Often used with @ExceptionHandler.

Example

```java
@ControllerAdvice
public class GlobalExceptionHandler {
}
```

Follow-up Questions

1. Difference between ControllerAdvice and ExceptionHandler?
2. Can it handle multiple exceptions?
3. Why is it useful?

Common Mistakes

- Thinking it handles exceptions automatically without methods.

Quick Revision

- Global handler
- All controllers
- Centralized logic
- Exception management
- Cleaner architecture

---

90. What is @ExceptionHandler?

Question

Interviewer: "What is @ExceptionHandler?"

Answer

@ExceptionHandler is used to define methods that handle specific exceptions. It can be used within a controller or inside a @ControllerAdvice class.

Detailed Explanation

- Handles specific exceptions.

- Returns custom responses.
- Works with ControllerAdvice.
- Improves API error handling.
- Reduces duplicate code.

Example

```java
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<String> handleException(
        ResourceNotFoundException ex) {

    return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
}
```

Follow-up Questions

1. Can one method handle multiple exceptions?
2. Where is it commonly used?
3. Difference from try-catch?

Common Mistakes

- Handling all exceptions with one generic method.

Quick Revision

- Specific exception handling
- Custom responses
- ControllerAdvice
- Cleaner APIs
- Error management

---

91. What is DTO?

Question

Interviewer: "What is a DTO?"

Answer

DTO (Data Transfer Object) is an object used to transfer data between layers of an application. It contains only the required fields and helps prevent exposing entity objects directly to clients.

Detailed Explanation

- Data Transfer Object.
- Transfers data between layers.
- Hides entity details.
- Improves security.
- Reduces unnecessary fields.
- Common in REST APIs.

Example

```java
public class EmployeeDTO {

    private Long id;
    private String name;

}
```

Follow-up Questions

1. Why not return entities directly?
2. DTO vs Entity?
3. How do you map DTOs?

Common Mistakes

- Returning JPA entities directly from APIs.
- Including sensitive fields.

Quick Revision

- Data transfer
- API response
- Security

- Lightweight objects
- Layer separation

---

92. Why do we use DTOs?

Question

Interviewer: "Why are DTOs important?"

Answer

DTOs help control the data exposed to clients, improve security, reduce payload size, and decouple API models from database entities.

Detailed Explanation

- Hides internal structure.
- Protects sensitive fields.
- Smaller response payload.
- Improves maintainability.
- Supports API versioning.
- Reduces coupling.

Example

Entity:

```java
public class Employee {
    private Long id;
    private String name;
    private String password;
}
```

DTO:

```java
public class EmployeeDTO {
    private Long id;
    private String name;
}
```

Follow-up Questions

1. What problems do DTOs solve?
2. How do DTOs improve security?
3. Have you used DTOs in your project?

Common Mistakes

- Exposing passwords through entities.
- Creating DTOs identical to entities.

Quick Revision

- Security
- Reduced payload
- Decoupling
- Maintainability
- API design

---

93. What is Lombok?

Question

Interviewer: "What is Lombok?"

Answer

Lombok is a Java library that automatically generates boilerplate code such as getters, setters, constructors, builders, equals(), hashCode(), and toString() at compile time.

Detailed Explanation

- Reduces boilerplate code.
- Uses annotations.
- Improves readability.
- Generates methods automatically.
- Increases productivity.

Example

```java
@Data
public class Employee {

    private Long id;
    private String name;

}
```

Follow-up Questions

1. How does Lombok work?
2. What annotations are commonly used?
3. Any disadvantages?

Common Mistakes

- Not installing Lombok plugin in IDE.
- Assuming generated code doesn't exist.

Quick Revision

- Boilerplate reduction
- Annotations
- Getter/Setter
- Constructors
- Builder support

---

94. What is @Getter?

Question

Interviewer: "What is @Getter in Lombok?"

Answer

@Getter automatically generates getter methods for fields at compile time, reducing manual code writing.

Detailed Explanation

- Generates getter methods.

- Reduces boilerplate.
- Can be applied to class or field.
- Compile-time generation.
- Improves readability.

Example

```java
@Getter
public class Employee {

    private String name;

}
```

Follow-up Questions

1. Can it be applied to fields?
2. Difference from @Data?
3. Does it generate setters?

Common Mistakes

- Expecting setter generation.

Quick Revision

- Generates getters
- Compile-time
- Less code
- Class level
- Field level

---

95. What is @Setter?

Question

Interviewer: "What is @Setter?"

Answer

@Setter automatically generates setter methods for fields at compile time.

Detailed Explanation

- Generates setters.
- Reduces manual coding.
- Works at class or field level.
- Improves readability.
- Part of Lombok.

Example

```java
@Setter
public class Employee {

    private String name;

}
```

Follow-up Questions

1. Difference from @Getter?
2. Can it be field-level?
3. Is it included in @Data?

Common Mistakes

- Using setters in immutable classes.

Quick Revision

- Generates setters
- Compile-time
- Lombok
- Class level
- Field level

96. What is @Data?

Question

Interviewer: "What does @Data provide?"

Answer

@Data is a Lombok annotation that generates getters, setters, toString(), equals(), hashCode(), and required constructors automatically.

Detailed Explanation

- Combines multiple annotations.
- Generates getters/setters.
- Generates toString().
- Generates equals/hashCode().
- Reduces boilerplate significantly.

Example

```java
@Data
public class Employee {

    private Long id;
    private String name;

}
```

Follow-up Questions

1. What annotations are included?
2. Difference from Getter/Setter?
3. When should @Data be avoided?

Common Mistakes

- Using @Data on JPA entities without understanding equals/hashCode effects.

Quick Revision

- Getter
- Setter
- toString
- equals
- hashCode

---

97. What is @Builder?

Question

Interviewer: "What is Lombok @Builder?"

Answer

@Builder implements the Builder Design Pattern automatically, allowing objects to be created step by step in a readable manner.

Detailed Explanation

- Builder Pattern support.
- Readable object creation.
- Supports optional fields.
- Reduces constructor complexity.
- Improves maintainability.

Example

```java
@Builder
public class Employee {

    private Long id;
    private String name;

}
```

Usage:

```java
```

```
Employee emp = Employee.builder()
        .id(1L)
        .name("John")
        .build();
```

Follow-up Questions

1. Why use Builder Pattern?
2. Builder vs Constructor?
3. Builder for immutable objects?

Common Mistakes

- Using large constructors instead of builders.

Quick Revision

- Builder Pattern
- Readable code
- Optional fields
- Less complexity
- Lombok support

---

98. What is @NoArgsConstructor?

Question

Interviewer: "What is @NoArgsConstructor?"

Answer

@NoArgsConstructor generates a constructor with no parameters automatically.

Detailed Explanation

- Creates default constructor.
- Useful for JPA entities.
- Reduces boilerplate.
- Compile-time generation.
- Lombok annotation.

Example

```java
@NoArgsConstructor
public class Employee {
}
```

Follow-up Questions

1. Why does JPA need it?
2. Difference from AllArgsConstructor?
3. Is constructor generated automatically?

Common Mistakes

- Forgetting JPA requires a no-argument constructor.

Quick Revision

- Default constructor
- JPA support
- Lombok
- No parameters
- Boilerplate reduction

---

99. What is @AllArgsConstructor?

Question

Interviewer: "What is @AllArgsConstructor?"

Answer

@AllArgsConstructor generates a constructor containing all fields of the class as parameters.

Detailed Explanation

- Constructor with all fields.
- Reduces manual coding.
- Useful for object creation.

- Compile-time generation.
- Lombok annotation.

Example

```java
@AllArgsConstructor
public class Employee {

    private Long id;
    private String name;

}
```

Generated:

```java
public Employee(Long id, String name) {
    this.id = id;
    this.name = name;
}
```

Follow-up Questions

1. Difference from NoArgsConstructor?
2. Can both be used together?
3. Does Builder replace it?

Common Mistakes

- Forgetting constructor parameter order.

Quick Revision

- All fields constructor
- Lombok
- Compile-time generation
- Less boilerplate
- Object creation

Most Important Spring Boot Fresher Questions (Must Memorize)

1. Spring vs Spring Boot ⭐⭐⭐⭐⭐
2. Dependency Injection ⭐⭐⭐⭐⭐
3. IoC ⭐⭐⭐⭐⭐
4. Bean Lifecycle ⭐⭐⭐⭐⭐
5. Bean Scopes ⭐⭐⭐⭐⭐
6. @Autowired ⭐⭐⭐⭐⭐
7. @Component / @Service / @Repository ⭐⭐⭐⭐⭐
8. @Controller vs @RestController ⭐⭐⭐⭐⭐
9. Spring MVC Request Flow ⭐⭐⭐⭐⭐
10. Global Exception Handling ⭐⭐⭐⭐⭐
11. DTO ⭐⭐⭐⭐⭐
12. Lombok Annotations ⭐⭐⭐⭐

Next: **Part 5 (Q100–Q136) — Spring Data JPA, Hibernate, Entity Mapping, JPQL, Cascade Types, Repository Methods.** This is the section where most Java Spring Boot interviewers spend the most time.

2026-06-22 22:15:49 **You Asked**

Next

ChatGPT gpt-5-5

Part 5 (Q100–Q115) — Spring Data JPA, Hibernate & Entity Mapping

---

100. What is JPA?

Question

Interviewer: "What is JPA?"

Answer

JPA (Java Persistence API) is a specification for managing relational data in Java applications. It provides a standard way to map Java objects to database tables and perform CRUD operations.

Detailed Explanation

- JPA is a specification, not an implementation.
- Used for Object Relational Mapping (ORM).
- Maps Java objects to database tables.
- Reduces JDBC boilerplate code.
- Supports annotations for mapping.
- Commonly used with Hibernate.

Example

```java
@Entity
public class Employee {
}
```

Follow-up Questions

1. Is JPA a framework?
2. What is ORM?
3. Which implementations support JPA?

Common Mistakes

- Saying JPA is Hibernate.
- Confusing specification and implementation.

Quick Revision

- Specification
- ORM
- Database mapping
- CRUD operations
- Works with Hibernate

---

101. What is Hibernate?

Question

Interviewer: "What is Hibernate?"

Answer

Hibernate is the most popular implementation of JPA. It provides ORM functionality and handles database interactions by converting Java objects into database records.

Detailed Explanation

- ORM framework.
- Implements JPA specification.
- Generates SQL automatically.
- Supports caching.
- Reduces JDBC coding.
- Handles object mapping.

Example

```java
Employee employee = new Employee();
entityManager.persist(employee);
```

Follow-up Questions

1. Is Hibernate a JPA implementation?
2. What are Hibernate advantages?
3. What is SessionFactory?

Common Mistakes

- Treating Hibernate and JPA as identical.

Quick Revision

- ORM framework
- JPA implementation
- SQL generation

- Object mapping
- Caching support

---

102. Difference between JPA and Hibernate?

Question

Interviewer: "What is the difference between JPA and Hibernate?"

Answer

JPA is a specification that defines standards for ORM, while Hibernate is an implementation of that specification. JPA defines what should be done, and Hibernate provides the actual implementation.

Detailed Explanation

JPA	Hibernate
Specification	Implementation
Standard API	ORM Framework
Vendor-independent	Vendor-specific
Defines rules	Implements rules

Example

```java

@Entity
public class Employee {
}
```

JPA annotation executed internally by Hibernate.

Follow-up Questions

1. Can JPA work without Hibernate?
2. Other JPA implementations?
3. Why use JPA over Hibernate directly?

Common Mistakes

- Saying both are the same.

Quick Revision

- JPA = Specification
- Hibernate = Implementation
- ORM support
- Standardized API
- Vendor independence

---

103. What is @Entity?

Question

Interviewer: "What is the purpose of @Entity?"

Answer

@Entity marks a Java class as a JPA entity and maps it to a database table. Each entity object represents a row in the table.

Detailed Explanation

- Maps class to table.
- Managed by JPA.
- Represents database records.
- Requires primary key.
- Used with @Id.

Example

```java
@Entity
public class Employee {

    @Id
    private Long id;

}
```

Follow-up Questions

1. Can an entity exist without @Id?
2. What table name is used by default?
3. What is entity lifecycle?

Common Mistakes

- Forgetting primary key field.

Quick Revision

- Table mapping
- Database row
- Requires @Id
- JPA managed
- ORM entity

---

104. What is @Table?

Question

Interviewer: "What is @Table annotation?"

Answer

@Table specifies the database table name associated with an entity. If not provided, JPA uses the class name as the table name by default.

Detailed Explanation

- Custom table mapping.
- Specifies table name.
- Supports schema definition.
- Works with @Entity.
- Improves database flexibility.

Example

```java
@Entity
@Table(name="employees")
```

```
public class Employee {
}
```

Follow-up Questions

1. Is @Table mandatory?
2. What happens if omitted?
3. Can schema be specified?

Common Mistakes

- Assuming @Table is always required.

Quick Revision

- Maps table
- Custom table name
- Optional annotation
- Used with Entity
- Database flexibility

---

105. What is @Id?

Question

Interviewer: "What is @Id in JPA?"

Answer

@Id marks a field as the primary key of an entity. It uniquely identifies each record in the database table.

Detailed Explanation

- Primary key annotation.
- Uniquely identifies rows.
- Required for entities.
- Used with GeneratedValue.
- Essential for persistence.

Example

```java
@Id
private Long id;
```

Follow-up Questions

1. Why is primary key needed?
2. Can composite keys exist?
3. What is GeneratedValue?

Common Mistakes

- Missing @Id in entity classes.

Quick Revision

- Primary key
- Unique identifier
- Entity requirement
- Database mapping
- Persistence support

---

106. What is @GeneratedValue?

Question

Interviewer: "What is @GeneratedValue?"

Answer

@GeneratedValue automatically generates primary key values for entities. It eliminates the need to manually assign IDs.

Detailed Explanation

- Auto-generates IDs.
- Works with @Id.
- Supports strategies.

- Simplifies inserts.
- Database-managed IDs.

Example

```java
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Follow-up Questions

1. What are generation strategies?
2. Difference between IDENTITY and AUTO?
3. Which strategy is commonly used?

Common Mistakes

- Generating IDs manually when auto-generation exists.

Quick Revision

- Auto ID generation
- Used with Id
- IDENTITY
- AUTO
- Sequence support

---

107. What is @Column?

Question

Interviewer: "What is @Column annotation?"

Answer

@Column maps a field to a database column and allows customization such as column name, length, nullable constraints, and uniqueness.

Detailed Explanation

- Column mapping.
- Custom column names.
- Length restrictions.
- Nullable configuration.
- Unique constraints.

Example

```java
@Column(name="employee_name",
        nullable=false,
        length=100)
private String name;
```

Follow-up Questions

1. Is @Column mandatory?
2. What happens if omitted?
3. Can uniqueness be enforced?

Common Mistakes

- Using wrong column names.

Quick Revision

- Field mapping
- Custom names
- Constraints
- Length control
- Optional annotation

---

108. What is @OneToOne?

Question

Interviewer: "Explain One-to-One mapping."

Answer

@OneToOne represents a relationship where one entity is associated with exactly one entity of another type.

Detailed Explanation

- One record ↔ One record.
- Foreign key relationship.
- Bidirectional or unidirectional.
- Common for profile details.
- Entity association.

Example

```java
@OneToOne
@JoinColumn(name="address_id")
private Address address;
```

Follow-up Questions

1. What is JoinColumn?
2. Bidirectional mapping?
3. Real-world example?

Common Mistakes

- Incorrect ownership side mapping.

Quick Revision

- One ↔ One
- Foreign key
- Address example
- JoinColumn
- Entity association

---

109. What is @OneToMany?

Question

Interviewer: "Explain One-to-Many mapping."

Answer

@OneToMany represents a relationship where one parent entity can have multiple child entities.

Detailed Explanation

- One parent.
- Multiple children.
- Common in departments and employees.
- Supports cascading.
- Collection-based mapping.

Example

```java
@OneToMany(mappedBy = "department")
private List<Employee> employees;
```

Follow-up Questions

1. What is mappedBy?
2. Bidirectional mapping?
3. Common use cases?

Common Mistakes

- Forgetting mappedBy.

Quick Revision

- One parent
- Many children
- Collection mapping
- mappedBy
- Cascading support

110. What is @ManyToOne?

Question

Interviewer: "Explain Many-to-One mapping."

Answer

@ManyToOne represents a relationship where multiple entities are associated with a single parent entity.

Detailed Explanation

- Many children.
- One parent.
- Foreign key stored here.
- Most common mapping.
- Supports joins.

Example

```java
@ManyToOne
@JoinColumn(name="department_id")
private Department department;
```

Follow-up Questions

1. Which side owns relationship?
2. Why use JoinColumn?
3. Relation with OneToMany?

Common Mistakes

- Not understanding owning side.

Quick Revision

- Many → One
- Foreign key
- Owning side

- Department example
- Common mapping

---

111. What is @ManyToMany?

Question

Interviewer: "Explain Many-to-Many mapping."

Answer

@ManyToMany represents a relationship where multiple entities can be associated with multiple entities of another type.

Detailed Explanation

- Many ↔ Many.
- Uses join table.
- Common for students and courses.
- Complex mapping.
- Supports bidirectional relationships.

Example

```java
@ManyToMany
@JoinTable(
    name="student_course"
)
private List<Course> courses;
```

Follow-up Questions

1. Why is join table needed?
2. Real-world example?
3. Bidirectional mapping?

Common Mistakes

- Forgetting join table configuration.

Quick Revision

- Many ↔ Many
- Join table
- Student-Course
- Complex relation
- Bidirectional support

---

112. What is @JoinColumn?

Question

Interviewer: "What is @JoinColumn?"

Answer

@JoinColumn specifies the foreign key column used to establish a relationship between entities.

Detailed Explanation

- Defines foreign key.
- Used in associations.
- Relationship owner.
- Supports custom names.
- Common with ManyToOne.

Example

```java
@JoinColumn(name="department_id")
```

Follow-up Questions

1. Difference from JoinTable?
2. When is it used?
3. What is owning side?

Common Mistakes

- Using wrong foreign key name.

Quick Revision

- Foreign key
- Relationship mapping
- Custom names
- Association support
- Ownership

---

113. What is @Transient?

Question

Interviewer: "What is @Transient?"

Answer

@Transient marks a field that should not be persisted to the database. JPA ignores such fields during database operations.

Detailed Explanation

- Excluded from persistence.
- Not mapped to columns.
- Runtime-only field.
- Useful for calculations.
- Ignored by JPA.

Example

```java
@Transient
private int age;
```

Follow-up Questions

1. Difference from transient keyword?
2. When should it be used?

3. Is column created?

Common Mistakes

- Expecting value to be stored in database.

Quick Revision

- Not persisted
- No column
- Runtime only
- Ignored by JPA
- Calculation fields

---

114. What is @Enumerated?

Question

Interviewer: "What is @Enumerated?"

Answer

@Enumerated is used to map Java enum values to database columns.

Detailed Explanation

- Maps enum fields.
- Supports STRING strategy.
- Supports ORDINAL strategy.
- Improves readability.
- Common for status fields.

Example

```java
@Enumerated(EnumType.STRING)
private Status status;
```

Follow-up Questions

1. STRING vs ORDINAL?

2. Which is recommended?

3. Why use enums?

Common Mistakes

- Using ORDINAL unnecessarily.

Quick Revision

- Enum mapping
- STRING
- ORDINAL
- Status fields
- Database storage

---

115. What is @Embeddable?

Question

Interviewer: "What is @Embeddable?"

Answer

@Embeddable is used to define a reusable class whose fields can be embedded into an entity without creating a separate table.

Detailed Explanation

- Reusable value object.
- No separate table.
- Embedded inside entities.
- Reduces duplication.
- Used with @Embedded.

Example

```java

@Embeddable
public class Address {
```

```
    private String city;
    private String state;

}
```

Follow-up Questions

1. Difference from Entity?
2. What is @Embedded?
3. Why use Embeddable?

Common Mistakes

- Expecting a separate table to be created.

Quick Revision

- Reusable object
- No table
- Embedded fields
- Value object
- Used with Embedded

High-Frequency JPA Questions So Far

⭐⭐⭐⭐⭐

- JPA vs Hibernate
- @Entity
- @Id
- @GeneratedValue
- @OneToMany
- @ManyToOne
- @JoinColumn

⭐⭐⭐⭐

- @Column
- @ManyToMany
- @Transient

- @Enumerated

Next: **Q116–Q136** covering:

- @Embedded
- Repository Methods (`save` , `findById` , `findAll` , `delete`)
- JPQL
- JPQL vs SQL
- Native Queries
- Cascade Types (ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH)

These are among the most commonly asked Spring Data JPA interview questions.

2026-06-22 22:16:49 **You Asked**

Next

ChatGPT gpt-5-5

Part 5 (Q116–Q136) — Repository Methods, JPQL & Cascade Types

---

116. What is @Embedded?

Question

Interviewer: "What is @Embedded in JPA?"

Answer

@Embedded is used to include an @Embeddable class inside an entity. The embedded object's fields become columns in the entity's table rather than creating a separate table.

Detailed Explanation

- Works with @Embeddable.
- No separate table created.
- Reuses common fields.
- Improves code organization.

- Embedded fields stored in parent table.

Example

```java
@Embeddable
public class Address {
    private String city;
    private String state;
}

@Entity
public class Employee {

    @Embedded
    private Address address;
}
```

Follow-up Questions

1. Difference between Entity and Embeddable?
2. Does @Embedded create a table?
3. Why use Embeddable objects?

Common Mistakes

- Expecting a separate table for embedded objects.

Quick Revision

- Used with @Embeddable
- No separate table
- Reusable fields
- Parent table storage
- Cleaner design

---

117. What are common JPA Repository methods?

Question

Interviewer: "What repository methods do you commonly use?"

Answer

Common JPA repository methods include save(), findById(), findAll(), delete(), deleteById(), existsById(), and count(). These methods provide CRUD functionality without writing SQL queries.

Detailed Explanation

- save() → Insert/Update.
- findById() → Fetch by ID.
- findAll() → Fetch all records.
- delete() → Remove entity.
- deleteById() → Remove by ID.
- existsById() → Check existence.
- count() → Count records.

Example

```java
employeeRepository.save(employee);
employeeRepository.findAll();
```

Follow-up Questions

1. Which method performs updates?
2. How do custom queries work?
3. What does existsById() return?

Common Mistakes

- Writing unnecessary SQL for basic CRUD.

Quick Revision

- save
- findById
- findAll
- delete
- count

118. What does save() do?

Question

Interviewer: "What does the save() method do?"

Answer

save() inserts a new entity if the ID doesn't exist, and updates an existing entity if the ID already exists.

Detailed Explanation

- Insert operation.
- Update operation.
- Managed by Hibernate.
- Returns saved entity.
- Most frequently used repository method.

Example

```java
Employee emp = new Employee();
employeeRepository.save(emp);
```

Follow-up Questions

1. How does save determine insert vs update?
2. What is returned by save()?
3. Is transaction required?

Common Mistakes

- Assuming save() only inserts data.

Quick Revision

- Insert
- Update
- Entity persistence
- Returns entity

- CRUD operation

---

119. What does findById() do?

Question

Interviewer: "What does findById() return?"

Answer

findById() retrieves an entity using its primary key and returns an Optional object to safely handle missing records.

Detailed Explanation

- Search by primary key.
- Returns Optional.
- Avoids NullPointerException.
- Common lookup operation.
- Database query execution.

Example

```java
Optional<Employee> employee =
        employeeRepository.findById(1L);
```

Follow-up Questions

1. Why Optional is returned?
2. What if ID doesn't exist?
3. Difference from getById()?

Common Mistakes

- Calling get() without checking Optional.

Quick Revision

- Search by ID
- Returns Optional

- Safe retrieval
- Primary key lookup
- CRUD method

---

120. What does findAll() do?

Question

Interviewer: "What is the purpose of findAll()?"

Answer

findAll() retrieves all records from the corresponding database table and returns them as a collection.

Detailed Explanation

- Fetches all entities.
- Returns List.
- Simple retrieval.
- Common in reporting.
- Auto-generated query.

Example

```java
List<Employee> employees =
        employeeRepository.findAll();
```

Follow-up Questions

1. Can pagination be used?
2. What is returned?
3. Is sorting supported?

Common Mistakes

- Using findAll() for huge datasets.

Quick Revision

- Fetch all

- List return type
- Auto-generated query
- Reporting use
- CRUD method

---

121. What does delete() do?

Question

Interviewer: "What does delete() do?"

Answer

delete() removes a specific entity object from the database.

Detailed Explanation

- Deletes entity.
- Requires entity object.
- Removes database record.
- Managed by Hibernate.
- Supports cascading.

Example

```java
employeeRepository.delete(employee);
```

Follow-up Questions

1. Difference from deleteById()?
2. Does cascading apply?
3. What happens if entity doesn't exist?

Common Mistakes

- Passing unmanaged entities.

Quick Revision

- Delete entity
- Remove record
- Uses object
- Hibernate managed
- CRUD method

---

122. What does deleteById() do?

Question

Interviewer: "What does deleteById() do?"

Answer

deleteById() deletes a record directly using its primary key without requiring the entity object.

Detailed Explanation

- Deletes using ID.
- Convenient method.
- No entity retrieval needed.
- Simple deletion.
- Common in REST APIs.

Example

```java
employeeRepository.deleteById(1L);
```

Follow-up Questions

1. Difference from delete()?
2. What if ID doesn't exist?
3. Which is more efficient?

Common Mistakes

- Assuming entity object is required.

Quick Revision

- Delete by ID
- No object needed
- Simple deletion
- Repository method
- CRUD operation

---

123. What does existsById() do?

Question

Interviewer: "What is existsById() used for?"

Answer

existsById() checks whether a record with a given primary key exists in the database and returns a boolean value.

Detailed Explanation

- Returns boolean.
- Checks existence.
- Avoids unnecessary fetch.
- Lightweight query.
- Validation use case.

Example

```java
boolean exists =
        employeeRepository.existsById(1L);
```

Follow-up Questions

1. Why not use findById()?
2. What is returned?
3. Is it efficient?

Common Mistakes

- Fetching full entity when existence check is enough.

Quick Revision

- Boolean result
- Existence check
- Efficient
- Validation
- Repository method

---

124. What does count() do?

Question

Interviewer: "What is the count() method?"

Answer

count() returns the total number of records available in the corresponding database table.

Detailed Explanation

- Counts rows.
- Returns long.
- Useful for statistics.
- Supports pagination.
- Efficient aggregation.

Example

```java
long total =
        employeeRepository.count();
```

Follow-up Questions

1. Return type?
2. Where is count used?

3. Can custom count queries be written?

Common Mistakes

- Using findAll().size() instead of count().

Quick Revision

- Row count
- Returns long
- Statistics
- Pagination
- Aggregation

---

125. What is JPQL?

Question

Interviewer: "What is JPQL?"

Answer

JPQL (Java Persistence Query Language) is an object-oriented query language used to query JPA entities rather than database tables.

Detailed Explanation

- Works on entities.
- Similar to SQL.
- Database-independent.
- Uses entity names.
- Supports joins and filtering.
- Converted to SQL by JPA provider.

Example

```java
@Query("SELECT e FROM Employee e")
List<Employee> findEmployees();
```

Follow-up Questions

1. Difference from SQL?
2. Difference from Native Query?
3. Why use JPQL?

Common Mistakes

- Using table names instead of entity names.

Quick Revision

- Entity-based query
- Object-oriented
- Database independent
- Uses entity names
- Converted to SQL

---

126. Difference between JPQL and SQL?

Question

Interviewer: "Compare JPQL and SQL."

Answer

JPQL works with entities and object fields, whereas SQL works directly with database tables and columns.

Detailed Explanation

JPQL	SQL
Entity based	Table based
Database independent	Database specific
Uses class names	Uses table names
Object-oriented	Relational

Example

JPQL:

```java
SELECT e FROM Employee e
```

SQL:

```sql
SELECT * FROM employees;
```

Follow-up Questions

1. Which is database-independent?
2. Which is faster?
3. When should SQL be preferred?

Common Mistakes

- Using table names inside JPQL.

Quick Revision

- JPQL → Entity
- SQL → Table
- JPQL portable
- SQL database-specific
- ORM support

---

127. Difference between JPQL and Native Query?

Question

Interviewer: "What is the difference between JPQL and Native Query?"

Answer

JPQL uses entity objects and is database-independent, while Native Queries use actual SQL syntax and database tables directly.

Detailed Explanation

JPQL	Native Query
Entity based	SQL based
Portable	Database specific
Easier maintenance	More control
ORM friendly	Direct SQL

Example

JPQL:

```java
@Query("SELECT e FROM Employee e")
```

Native Query:

```java
@Query(
 value="SELECT * FROM employees",
 nativeQuery=true
)
```

Follow-up Questions

1. When would you use Native Query?
2. Which is portable?
3. Performance differences?

Common Mistakes

- Using Native Query unnecessarily.

Quick Revision

- JPQL = Entity
- Native = SQL
- Portable vs Specific
- ORM support
- Direct database access

128. What are the advantages of JPQL?

Question

Interviewer: "Why would you use JPQL?"

Answer

JPQL provides database independence, better integration with JPA entities, improved readability, and object-oriented querying without dealing directly with database tables.

Detailed Explanation

- Database independent.
- Entity-based querying.
- Better readability.
- Easier maintenance.
- ORM-friendly.
- Reduced SQL dependency.

Example

```java
@Query("SELECT e FROM Employee e WHERE e.salary > :salary")
```

Follow-up Questions

1. Why not use SQL?
2. What is portability?
3. Can joins be used?

Common Mistakes

- Using SQL syntax inside JPQL.

Quick Revision

- Entity queries
- Portable
- Readable

- ORM support
- Easy maintenance

---

129. Have you used JPQL in your project?

Question

Interviewer: "Have you used JPQL in your project?"

Answer

Yes. I used JPQL to fetch filtered data from entities where derived query methods were not sufficient. For example, retrieving employees based on department, salary range, or status using custom queries.

Detailed Explanation

- Custom retrieval logic.
- Complex filtering.
- Entity-based queries.
- Better readability.
- Used with @Query annotation.

Example

```java
@Query("SELECT e FROM Employee e WHERE e.department = :dept")
List<Employee> findByDepartment(String dept);
```

Follow-up Questions

1. Why not use findByDepartment()?
2. Did you use joins?
3. Did you use Native Queries?

Common Mistakes

- Claiming project usage without examples.

Quick Revision

- Custom queries
- @Query annotation
- Filtering
- Entity-based
- Real project example

---

130. What are Cascade Types?

Question

Interviewer: "What are Cascade Types in JPA?"

Answer

Cascade Types define how operations performed on a parent entity should automatically propagate to related child entities.

Detailed Explanation

- Parent-child relationship.
- Automates entity operations.
- Reduces manual handling.
- Common with associations.
- Supports multiple cascade options.

Example

```java
@OneToMany(cascade = CascadeType.ALL)
private List<Employee> employees;
```

Follow-up Questions

1. What is CascadeType.ALL?
2. Difference between PERSIST and MERGE?
3. When should REMOVE be used?

Common Mistakes

- Using ALL everywhere without understanding impact.

Quick Revision

- Parent-child
- Automatic propagation
- Entity relationships
- Simplifies operations
- Multiple types

---

131. What is CascadeType.ALL?

Question

Interviewer: "What does CascadeType.ALL do?"

Answer

CascadeType.ALL applies all cascade operations including PERSIST, MERGE, REMOVE, REFRESH, and DETACH to child entities.

Example

```java
@OneToMany(cascade = CascadeType.ALL)
private List<Employee> employees;
```

Quick Revision

- All cascade operations
- Parent controls children
- Simplifies management
- Powerful option
- Use carefully

---

132. What is CascadeType.PERSIST?

Question

Interviewer: "What is CascadeType.PERSIST?"

Answer

When a parent entity is saved, child entities are automatically saved as well.

Example

```java
cascade = CascadeType.PERSIST
```

Quick Revision

- Save parent
- Save child
- Insert propagation
- New entities
- Automatic persistence

---

133. What is CascadeType.MERGE?

Question

Interviewer: "What is CascadeType.MERGE?"

Answer

When a parent entity is updated, associated child entities are automatically updated.

Example

```java
cascade = CascadeType.MERGE
```

Quick Revision

- Update parent
- Update child
- Merge operation

- Synchronization
- Entity updates

---

134. What is CascadeType.REMOVE?

Question

Interviewer: "What is CascadeType.REMOVE?"

Answer

When a parent entity is deleted, associated child entities are automatically deleted as well.

Example

```java
cascade = CascadeType.REMOVE
```

Quick Revision

- Delete parent
- Delete child
- Cascading delete
- Relationship cleanup
- Use carefully

---

135. What is CascadeType.REFRESH?

Question

Interviewer: "What is CascadeType.REFRESH?"

Answer

REFRESH reloads the entity state from the database and applies the refresh operation to child entities.

Example

```java
```

```
cascade = CascadeType.REFRESH
```

Quick Revision

- Reload data
- Sync with DB
- Refresh child entities
- Discard local changes
- Database state

---

136. What is CascadeType.DETACH?

Question

Interviewer: "What is CascadeType.DETACH?"

Answer

DETACH removes an entity from the persistence context, and the detach operation is propagated to associated child entities.

Detailed Explanation

- Removes entity from persistence context.
- No longer managed by EntityManager.
- Changes won't be tracked.
- Propagates to child entities.
- Useful in advanced JPA scenarios.

Example

```java
cascade = CascadeType.DETACH
```

Follow-up Questions

1. What is persistence context?
2. Difference between REMOVE and DETACH?

3. When is DETACH used?

Common Mistakes

- Confusing DETACH with DELETE.

Quick Revision

- Unmanaged entity
- Persistence context
- No tracking
- Child propagation
- Advanced JPA feature

Most Asked JPA Interview Questions

⭐⭐⭐⭐⭐

- JPA vs Hibernate
- @Entity
- @OneToMany
- @ManyToOne
- JPQL
- JPQL vs SQL
- Cascade Types
- save() vs findById()

⭐⭐⭐⭐

- @ManyToMany
- @Embedded
- Native Query
- Repository Methods

For a fresher Spring Boot interview, if you master Q100–Q136 plus the earlier Spring Boot section, you'll cover the majority of JPA questions typically asked in service-based company interviews such as Cognizant, TCS, Accenture, Capgemini, Wipro, Infosys, and similar organizations.